République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene



Faculté de Génie Electrique

Département d'Electronique

Domaine Sciences et Technologie

Filière : Electronique

Mémoire de Master 2

Spécialité : Electronique des systèmes embarqués

Thème :

Conception d'un système de santé connecté avec monitoring des paramètres vitaux en temps réel

Présenté par : BELKACEM Mohamed Amine

Soutenu le : 22/06/2025

Promotion : Juin 2025

الملخص

الهدف من هذه الاطروحة هو تصميم نظام صحي متصل يسمح بمراقبة المعلمات الحيوية في الوقت الفعلي، مثل النشاط لكهربائي ودرجة حرارة الجسم ونوعية الهواء. يعتمد النظام على استخدام AD8232 للقلب لقياس تخطيط القلب ، بالإضافة إلى متحكم يسمح هذا الجهاز بمراقبة حالة .Raspberry Pi 5 لجمع البيانات ونقلها إلى خادم محلي مستضاف على ESP32 مستشعر في (IoT) المريض عن بعد ويعزز الكشف المبكر عن التشوهات. وهو يوضح بشكل ملموس تكامل تقنيات إنترنت الأشياء المجال الطبي، مما يوفر رعاية أكثر ذكاءً وسهولة في الوصول إليها.

Résume

Ce mémoire a pour objectif de concevoir un système de santé connecté permettant la surveillance en temps réel des paramètres vitaux, tels que l'activité électrique du cœur, la température corporelle et la qualité de l'air. Le système repose sur l'utilisation du capteur AD8232 pour la mesure de l'ECG, ainsi que du microcontrôleur ESP32 pour la collecte et la transmission des données vers un serveur local hébergé sur un Raspberry Pi 5. Ce dispositif permet une surveillance à distance de l'état du patient et favorise une détection précoce des anomalies. Il illustre concrètement l'intégration des technologies de l'Internet des Objets (IoT) dans le domaine médical, au service d'une prise en charge plus intelligente et plus accessible.

Abstract

This thesis aims to develop a connected health system for real-time monitoring of vital parameters, such as heart electrical activity, body temperature, and air quality. The system uses the AD8232 sensor to measure ECG signals and relies on an ESP32 microcontroller to gather data from medical sensors and send it to a local server hosted on a Raspberry Pi 5. This setup enables remote patient monitoring and supports early detection of health anomalies. The project demonstrates a practical integration of Internet of Things (IoT) technologies in healthcare and highlights the potential of smart monitoring systems in improving modern medical care.

Table des matières:

Introduction générale 1

Chapitre 1 : Introduction au système cardiovasculaire et à l'électrocardiogramme 3

1.1 Introduction 4

1.2 Le système cardiovasculaire humain 4

1.2.1 Structure générale et rôles physiologiques 4

1.2.2 Constitution du système cardiovasculaire 4

1.3 Le cœur : moteur electrique du systeme 5

1.3.1 Anatomie et fonctionnement du cœur humain 5

1.4 Le cycle cardiaque 6

1.5 L'activité électrique de cœur 1.5.1 Origine et propagation du signal électrique 7

1.5.1 Origine et propagation du signal électrique 8

1.5.2 Le système de conduction cardiaque 8

1.6 L'électrocardiogramme (ECG) : principes et applications médicales 9

1.6.1 Principe de fonctionnement de l'ECG : 9

1.6.2 Lecture et interprétation du tracé ECG : 9

1.6.3 Intérêt clinique de l'analyse ECG 11

1.6.4 Types d'ECG et dispositifs d'acquisition 11

1.6.5 Applications medicale de l'ECG 12

1.7 Conclusion 13

Chapitre 2: Internet des Objets (IoT) et son application dans le domaine médical 14

2.1 Introduction 15

2.2 Internet des Objets Médicaux (IoMT) 16

2.2.1 Domaines d'application du IoMT 16

2.2.2 Exemples de dispositifs IoMT 17

2.2.3 Bénéfices du IoMT : 17

2.3 Les serveurs dans l'architecture IoT 17

2.3.1 Rôle des serveurs dans l'IoT 18

2.3.2 Serveur Flask : un microframework léger adapté à l'IoT 19

2.4 Les bases de données dans les systèmes IoT 20

2.4.1 Rôle et importance des bases de données en IoT 20

2.4.2 SQLite : une base de données légère adaptée aux systèmes embarqués 20

2.4.3 Interaction entre serveur et base de données 21

2.5 Les interfaces utilisateurs dans l'IoT médical 22

2.5.1 Les interfaces WEB 22
Technologies utilisées des interfaces WEB : HTML, CSS, JavaScript 22
2.5.2 Interaction avec le serveur Flask 22
2.5.3 Visualisation des données en temps réel avec Chart.js 23
2.5.4 Application mobile 23
2.5.5 Importance de la mobilité dans le suivi médical 23
2.5.6 Présentation de Flutter 24
2.5.7 Architecture d'une application mobile Flutter pour l'IoT médical 24
2.5.8 Synchronisation avec le serveur Flask 24
2.6 Protocoles de communication : HTTP et Wi-Fi 25
2.6.1 Le Wi-Fi en IOT 25
2.6.2 Le protocole http en IOT 26
2.7 Conclusion : 26
Chapitre 3 Conception et réalisation de notre système du sante connecte 27
3.1 Introduction 28
3.2 Architecture fonctionnelle du système : 29
3.3 Partie matérielle : 30
3.3.1 Microcontrôleurs et micro-ordinateur embarqués 30
a) Carte de développement ESP32 30
b) Le Raspberry Pi 5 31
3.3.2Les capteurs : 33
a) Module ECG AD8232 33
b) Capteur SpO2 et fréquence cardiaque MAX30102 35
c)Le capteur de qualité de l'air MQ-135 36
d) Le capteur de température et d'humidité DHT22 37
3.3.4 Tableau des connexions entre les capteurs et l'ESP32 : 39
3.3.5Conversion des valeurs brutes ECG en unités physiques exploitables 40
3.3.6 Traitement des données des autres capteurs environnementaux : 41
3.3.7 Schéma de câblage 41
3.3.8 Schéma électrique: 42
3.3.9 Conception du circuit imprimé (PCB) 43
3.4 Partie Logiciels 44
3.4.1 Protocoles de communication 44
3.4.2 Transmission des données via le Protocol HTTP 46
3.4.3 Système d'exploitation utilise pour l'hébergement du serveur 46
3.4.4 Serveur Flask 47
a) Installation et développement de l'environnement du serveur 47
b) Création de l'arborescence du projet 47
c) Création des routes API 48
d) Mise en marche du serveur Flask sur Raspberry Pi 49

3.4.5 Mise en place de la base de données SQLite 50
Manipulation de la base de données SQLite : 51
3.4.6 Développement de la page web (interface cliente) 52
a. Choix des technologies 52
b. Structure de la page 52
c. Affichage en temps réel 53
d. Interconnexion entre les composants..... 54
3.4.7 Développement de l'application mobile Flutter 55
Notifications et alertes en temps réel dans l'application mobile Flutter 56
3.5 Passage du serveur local en réseau étendu (WAN) avec Ngrok 57
3.5.1 Transformatuin dun serveur LAN en serveur WAN 57
3.5.2 Comparaison LAN vs WAN 58
3.5.3 Sécurité et exposition des données 59
3.6 Test et Validation du fonctionnement global du système 60
Tests côté serveur Flask et base SQLite (Raspberry Pi 5)..... 61
Tests d'affichage sur la page web HTML/JS..... 62
Tests de l'application mobile Flutter..... 62
Tests d'accessibilité via Ngrok 63
3.7 Resultat et conclusion ……..64

Conclusion générale 66
BIBLIOGRAPHIE: 67
Annexe……..……69

Liste des figures :

Figure 1.1 : Voies circulatoires du système cardiovasculaire 5

Figure 1.2 : Anatomie du cœur humain 6

Figure 1.3 : Les cinq différentes phases du cycle cardiaque 7

Figure 1.4 : Le système de conduction cardiaque 8

Figure 1.5 : Moniteur des signes vitaux……..... 9

Figure 1.6 : ECG d'un rythme cardiaque normal 11

Figure 1.7 : Utilisation du signal ECG par différents acteurs médicaux 12

Figure 2.1 : Vue d'ensemble d'un système de santé connecté 15

Figure 2.2 : Écosystème médical interconnecté 16

Figure 2.3 : Fonctionnement d'un serveur dans un système IoT médical 18

Figure 2.4 : Architecture fonctionnelle d'un serveur Flask dans un système IoT ……19

Figure 2.5 : Structure montrant l'interaction entre le serveur et la base de données.....21

Figure 2.6 : Exemple d'interface avec Chart.js 23

Figure 2.7 : Les différents protocoles de communication en IoT 25

Figure 3.1 : Organigramme fonctionnel du système de santé connecté 29

Figure 3.2 : Carte de développement ESP32 30

Figure 3.3 : Carte Raspberry Pi 5 31

Figure 3.4 : Capteur ECG AD8232 33

Figure 3.5 : Différents placements des électrodes sur le corps .....…34

Figure 3.6 : Module du capteur MAX30102 face avant et arrière .....…36

Figure 3.7 : Capteur de qualité de l'air MQ135 37

Figure 3.8 : Capteur de température et d'humidité DHT22 38

Figure 3.9 : Schéma de câblage complet du système de santé connecté 42

Figure 3.10 : Schéma électrique complet du système de santé connecté 42

Figure 3.11 : Schéma du circuit imprimé (PCB) conçu sous EasyEDA 43

Figure 3.12 : Installation de Raspberry Pi OS via Raspberry Pi Imager .....…47

Figure 3.13 : Arborescence du projet Flask sur le Raspberry Pi 5 48

Figure 3.14 : Code serveur de la route /ecg 49

Figure 3.15 : Mise en marche du serveur 50

Figure 3.16 : Contenu du fichier csv de la base de donnée des capteurs 51

Figure 3.17 : Consultation de la base de données via le terminal 51

Figure 3.18 : Vue d'ensemble de la page web 53

Figure 3.19 : Graphe de l'ECG de la page web 54

Figure 3.20 : Aperçu d'application mobile Flutter en phase de développement 56

Figure 3.21 : Fenêtre pour modifier les seuils des capteurs 57

Figure 3.22 : Mise en marche de Ngrok à partir du terminal 58

Figure 3.23 : Tests du montage réel dans un lab d'essai 60

Figure 3.24 : Données envoyées à partir de l'ESP32 sur Serial Monitor 61

Figure 3.25 : Aperçu du signal ECG non filtré sur Serial Plotter 61

Figure 3.26 : Résultat de l'envoi des requêtes HTTP sur le serveur 61

Figure 3.27 : Exemple d'affichage de la courbe température sur la page web 62

Figure 3.28 : Page « À propos » de l'application Flutter 63

Figure 3.29 : Exemple d'affichage de la courbe humidité dans l'application Flutter..... 63

Figure 3.30 : Icône de l'application mobile Flutter 63

Figure 3.31 : Connexion du serveur local à Internet via Ngrok 64

Liste des tableaux :

Tableau 1.1 : Principaux composants de l'ECG et leur signification clinique 10

Tableau 2.1 : Exemples de dispositifs IoMT et leurs fonctions principales 17

Tableau 2.2 : Comparaison des avantages et inconvénients du protocole HTTP dans un contexte IoT…….….. 26

Tableau 3.1 : Avantages et inconvénients de la Raspberry Pi 5 32

Tableau 3.2 : Avantages et inconvénients du capteur AD8232 34

Tableau 3.3 : Connexions entre les capteurs et l'ESP32 39

Tableau 3.4 : Comparaison des technologies de communication sans fil utilisées en systèmes embarqués 45

Tableau 3.5 : Comparaison des modes d'accès au serveur : LAN local vs WAN 59

Listes des abréviations :

Acronymes	Signification
API	Application Programming Interface
BPM	Battements Par Minute
CPU	Central Processing Unit
CSV	Comma-Separated Values
DHT22	Digital Humidity and Temperature sensor (modèle 22)
ECG	Électrocardiogramme
ESP32	Espressif Systems 32-bit microcontroller
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
I^2C	Inter-Integrated Circuit
IoT	Internet of Things
IoMT	Internet of Medical Things
IDE	Integrated Development Environment
LED	Light Emitting Diode
MAX30102	Capteur de fréquence cardiaque et SpO_2
MQ	Metal Oxide Semiconductor (capteur de gaz type MQ)
PCB	Printed Circuit Board
RAM	Random Access Memory
SpO_2	Saturation pulsée en oxygène
Wi-Fi	Wireless Fidelity
µC	Microcontrôleur

Introduction générale

Au fil des dernières décennies, le domaine de la santé a profondément évolué grâce aux avancées technologiques. On assiste aujourd'hui à l'émergence de dispositifs connectés et de systèmes intelligents qui transforment la manière dont les soins sont dispensés et les patients suivis. L'Internet des objets (IoT), les systèmes embarqués et l'intelligence artificielle (IA) jouent un rôle central dans cette transformation. Ces technologies permettent notamment de surveiller à distance les paramètres vitaux, d'analyser les données en temps réel, et de soutenir les professionnels de santé dans leur prise de décision.

Dans ce contexte en pleine mutation, ce mémoire s'intéresse à la conception et au développement d'un système de santé connecté, basé sur des technologies modernes comme le microcontrôleur ESP32 et le capteur ECG AD8232. L'objectif est de créer une solution de télémédecine à la fois accessible, fiable et sécurisée, capable d'assurer une surveillance continue des signes vitaux des patients. Ce projet s'inscrit pleinement dans la dynamique actuelle de la santé connectée, en proposant un outil adaptable aux besoins du terrain tout en respectant les contraintes de performance et de sécurité propres aux environnements médicaux.

Le premier chapitre, intitulé Introduction au système cardiovasculaire et à l'électrocardiogramme, présente de manière détaillée l'anatomie et le fonctionnement du cœur, les mécanismes de l'activité électrique cardiaque, ainsi que le principe et l'interprétation du tracé ECG. L'objectif de ce chapitre est d'apporter les bases biomédicales nécessaires à la compréhension de la surveillance cardiaque dans les systèmes de santé connectés,

Le deuxième chapitre, Internet des Objets (IoT) appliqué au domaine médical, se concentre sur les concepts fondamentaux de l'Internet des Objets (IoT) appliqués au domaine médical, avec un accent particulier sur l'IoMT (Internet of Medical Things). Le chapitre détaille le rôle des serveurs embarqués (notamment Flask), la gestion des bases de données (comme SQLite), ainsi que les interfaces utilisateur développées pour la visualisation et l'interaction avec les données (applications web et mobiles). Ces éléments constituent l'architecture logicielle sur laquelle repose le système de monitoring en temps réel présenté dans les chapitres suivants.

Le troisième chapitre, intitulé Conception et réalisation du système, expose l'architecture complète du système que nous avons développé, de la collecte des données jusqu'à leur affichage. Nous y décrivons la programmation embarquée sur l'ESP32, l'intégration des capteurs, le stockage

et le traitement des données, ainsi que le déploiement du serveur Flask et de l'interface utilisateur mobile.

En conclusion, ce mémoire a permis de concevoir et de mettre en œuvre un système complet de surveillance des signaux physiologiques, centré sur l'électrocardiogramme (ECG), en exploitant les possibilités offertes par les technologies embarquées et les communications sans fil. De l'acquisition des données par capteurs jusqu'à leur visualisation en temps réel sur des interfaces modernes (web et mobile).

Chapitre 1 : Introduction au système cardiovasculaire et à l'électrocardiogramme

1.1 Introduction

Les appareils de mesure à distance sont de plus en plus utilisés au service de la médecine préventive, de la télémédecine et de la gestion des maladies chroniques. Ces appareils mesurent en continu des paramètres vitaux comme la fréquence cardiaque, la pression artérielle et même la saturation en oxygène. Parmi ces mesures, l'analyse électrique du cœur par électrocardiogramme (ECG) est essentielle sur le plan du dépistage anticipé des troubles cardiaques. Ce chapitre vise à poser les bases théoriques nécessaires à la compréhension du système de santé connecté développé.

1.2 Le système cardiovasculaire humain

1.2.1 Structure générale et rôles physiologiques

Le système cardiovasculaire est un réseau complexe et indispensable qui assure la circulation du sang dans tout le corps humain. Il joue un rôle fondamental dans le transport de l'oxygène, des nutriments, des hormones, ainsi que dans l'élimination des déchets métaboliques. Le bon fonctionnement est vital au maintien de l'homéostasie et à la survie des cellules.

1.2.2 Constitution du système cardiovasculaire :

Le système cardiovasculaire est principalement composé de trois éléments :

- Le cœur,
- Les vaisseaux sanguins (artères, veines et capillaires),
- Et le sang.

Le cœur fonctionne comme une pompe musculaire centrale, propulsant le sang à travers un circuit fermé. Les artères acheminent le sang oxygéné du cœur vers les organes et tissus, tandis que les veines ramènent le sang désoxygéné vers le cœur.

Les capillaires, qui sont les vaisseaux les plus fins du réseau, permettent les échanges entre le sang et les cellules : apport d'oxygène et de nutriments, et élimination du dioxyde de carbone et des déchets.
Le système cardiovasculaire comme montré dans la figure 1.1 est divisé en deux grandes circulations :

- La circulation pulmonaire, qui assure l'échange gazeux dans les poumons, en transportant le sang désoxygéné du cœur vers les poumons puis en ramenant le sang oxygéné vers le cœur.

- La circulation systémique, qui distribue le sang oxygéné à l'ensemble des tissus et organes du corps. [1]

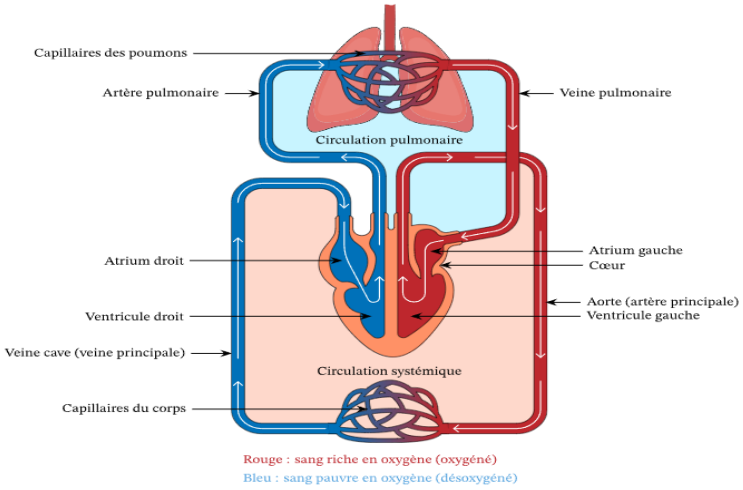


Figure 1.1 : Voies circulatoires du système cardiovasculaire

1.3 Le cœur : moteur électrique du système

Le cœur, organe central de ce système est doté d'un muscle spécifique, le myocarde, capable de contractions rythmiques et involontaires. Ces contractions sont commandées par un système électrique interne, garantissant la coordination des battements cardiaques et l'efficacité du pompage sanguin.

La compréhension de la structure et du fonctionnement du système cardiovasculaire est indispensable pour appréhender les bases du monitoring des paramètres vitaux, notamment la fréquence cardiaque et les anomalies détectables par électrocardiogramme (ECG). Ces connaissances constituent donc un socle essentiel à l'étude des systèmes de santé connectés dédiés à la surveillance en temps réel des patients. [2]

1.3.1 Anatomie et fonctionnement du cœur humain

Le cœur humain est un muscle creux au rôle fondamental dans la circulation sanguine. Placé au centre du système cardiovasculaire, il agit comme une pompe capable de maintenir en mouvement le sang dans tout l'organisme. Grâce à ce mécanisme, l'oxygène, les nutriments essentiels et les déchets métaboliques peuvent être transportés efficacement à travers le réseau vasculaire.
Sur le plan anatomique, le cœur est divisé en quatre cavités : deux oreillettes (droite et gauche) en partie supérieure, et deux ventricules (droit et gauche) en partie inférieure. Ces compartiments sont séparés par des cloisons musculaires, le septum interauriculaire et le septum interventriculaire qui

empêchent le mélange du sang entre les deux côtés du cœur. Pour garantir une circulation fluide et toujours dans le bon sens, le cœur est également équipé de valves : la tricuspide, la mitrale, la pulmonaire et l'aortique. Celles-ci agissent comme des clapets anti-retour, empêchant le sang de refluer à contre-courant.

Le côté droit du cœur prend en charge le sang pauvre en oxygène, en provenance des tissus corporels, via les veines caves. Ce sang est alors envoyé vers les poumons par l'intermédiaire de l'artère pulmonaire, où il est réoxygéné. Une fois enrichi en oxygène, il revient au côté gauche du cœur par les veines pulmonaires. De là, il est propulsé dans l'aorte, pour être distribué à l'ensemble des organes et tissus du corps. [3]

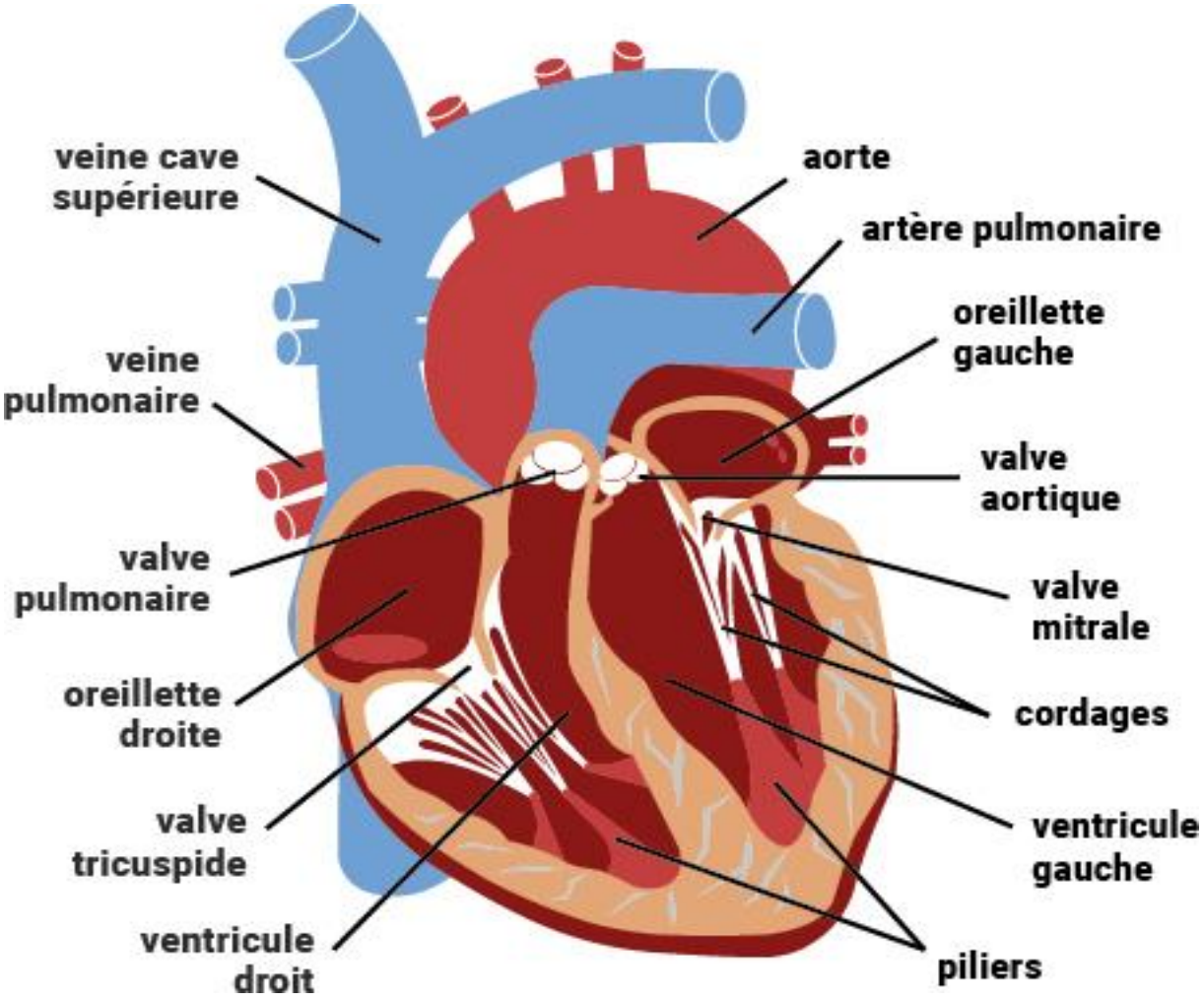


Figure 1.2 : Anatomie du cœur humain

1.4 Le cycle cardiaque

Le fonctionnement du cœur repose sur un cycle composé de deux phases principales :

- **La systole** : phase de contraction musculaire qui permet l'éjection du sang des ventricules.[4]

- **La diastole** : phase de relâchement qui permet le remplissage des oreillettes et des ventricules.[5]

Ce cycle comme on peut le voir sur la figure 1.3 est contrôlé par un système de conduction électrique intrinsèque, chargé de générer et de propager l'influx électrique nécessaire à la contraction coordonnée du muscle cardiaque. En réalité, le cycle cardiaque peut être subdivisé de manière plus détaillée en cinq phases successives :

1. **Remplissage ventriculaire**
2. **Contraction isovolumétrique**
3. **Éjection rapide du sang**
4. **Éjection réduite (ou lente)**
5. **Relaxation isovolumétrique**

Ces différentes phases assurent une circulation sanguine efficace et rythmée, synchronisée par l'activité électrique du cœur visible sur un électrocardiogramme (ECG).[6]

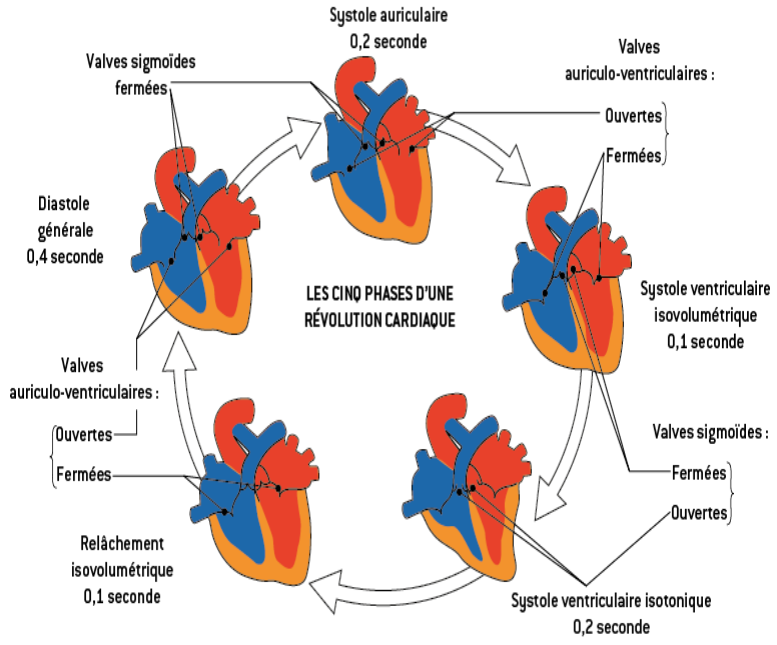


Figure 1.3 : Les cinq différentes phases du cycle cardiaque

1.5 L'activité électrique de cœur

1.5.1 Origine et propagation du signal électrique

Le bon fonctionnement du cœur repose sur un mécanisme électro physiologique complexe qui permet une contraction coordonnée des différentes cavités cardiaques. Cette activité électrique, indispensable à la circulation sanguine, est générée spontanément par le tissu cardiaque et se propage selon un chemin bien défini. Elle est à l'origine du signal mesurable par l'électrocardiogramme. [7]

1.5.2 Le système de conduction cardiaque

Le cœur possède un système de conduction électrique intrinsèque**,** composé de cellules spécialisées capables de générer et de propager des impulsions électriques assurant la contraction coordonnée du muscle cardiaque. Ce système, comme indiqué dans la figure 1.4 en-dessous, comprend les structures principales suivantes, organisées selon le cheminement de l'influx électrique :

1. Le nœud sinusal (ou sino-auriculaire), situé dans la paroi de l'oreillette droite, est le centre rythmogène principal. Il génère spontanément des potentiels d'action à une fréquence de 60 à 100 battements par minute, agissant ainsi comme le pacemaker naturel du cœur.
2. Le signal électrique se propage ensuite aux deux oreillettes, provoquant leur contraction.
3. L'influx atteint ensuite le nœud auriculo-ventriculaire (nœud AV)**,** qui joue un rôle de relais et ralentit légèrement la transmission, laissant le temps aux oreillettes de se vider complètement dans les ventricules.
4. Le signal est ensuite transmis au faisceau de His**,** qui traverse le septum interventriculaire, puis se divise en deux branches (droite et gauche) pour aboutir aux fibres de Purkinje**,** responsables de la contraction rapide et synchronisée des ventricules.

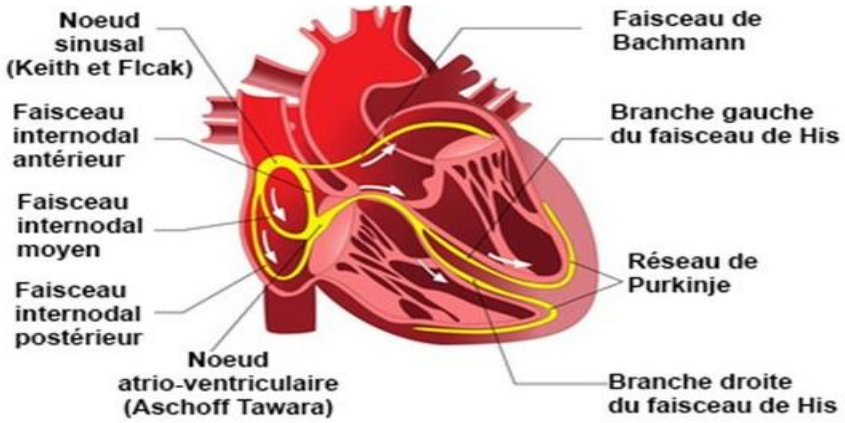


Figure 1.4 : Le système de conduction cardiaque

1.6 L'électrocardiogramme (ECG) : principes et applications médicales

1.6.1 Principe de fonctionnement de l'ECG :

L'électrocardiogramme (ECG) est une représentation graphique de l'activité électrique du muscle cardiaque au cours du temps. Cette activité électrique est à l'origine des contractions coordonnées du cœur, indispensables à la circulation sanguine. Le cœur génère spontanément des impulsions électriques grâce au nœud sinusal, situé dans l'oreillette droite, qui agit comme un stimulateur naturel.

Lorsque ces impulsions se propagent à travers les tissus cardiaques, elles créent de légères différences de potentiel mesurables à la surface du corps. Des électrodes positionnées sur la peau permettent de capter ces signaux. Ces derniers sont ensuite amplifiés, filtrés, puis enregistrés sous forme d'un tracé graphique qui reflète l'activité cardiaque en temps réel.

L'ECG constitue un outil fondamental en médecine pour le diagnostic des troubles du rythme cardiaque, des infarctus du myocarde, et d'autres anomalies cardiovasculaires. Grâce aux progrès technologiques, il peut désormais être mesuré à l'aide de dispositifs compacts et embarqués, facilitant ainsi sa surveillance en continu, y compris en dehors des environnements hospitaliers. [8]



Figure 1.5 : Moniteur des signes vitaux

1.6.2 Lecture et interprétation du tracé ECG :

Le signal ECG est composé de plusieurs ondes caractéristiques, chacune correspondant à une phase

précise de l'activité électrique du cœur. On distingue notamment l'onde P, le complexe QRS et l'onde T, qui traduisent respectivement l'activation des oreillettes, la dépolarisation des ventricules, et leur repolarisation. L'analyse de ces différentes composantes permet de détecter d'éventuelles anomalies cardiaques, telles que des arythmies, un bloc auriculo-ventriculaire ou un infarctus. [9]

Le tableau 1.1 ci-dessous présente les principales composantes du tracé ECG et leur signification physiologique.

Tableau 1.1 : Principaux composant de l'ECG et leur signification clinique

Composant de l'ECG	Description
Onde P	Dépolarisation des oreillettes, initiée par le nœud sinusal.
Intervalle PR	Temps de conduction entre oreillettes et ventricules via le nœud AV ; anomalies = bloc ou pré-excitation.
Complexe QRS	Dépolarisation des ventricules ; sa durée et sa forme renseignent sur la conduction intraventriculaire.
Segment ST	Phase de dépolarisation complète des ventricules ; anomalies = infarctus ou ischémie.
Onde T	Repolarisation des ventricules ; morphologie anormale = troubles électrolytiques ou ischémie.
Intervalle QT	Représente toute l'activité électrique ventriculaire ; un allongement peut entraîner des arythmies graves.

L'analyse fine de ces composantes, qui sont illustre dans la figure 1.6 permet de détecter diverses anomalies cardiaques et constitue un outil central dans la surveillance connectée de la santé.

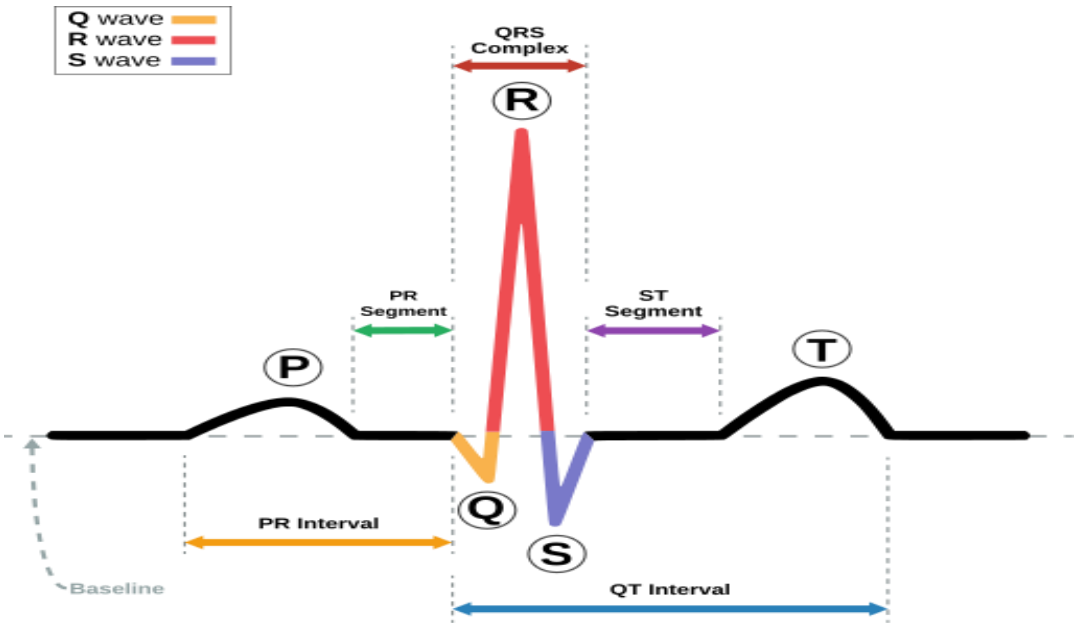


Figure 1.6 : ECG d'un rythme cardiaque normal [10]

1.6.3 Intérêt clinique de l'analyse ECG :

L'analyse de ces différents paramètres permet de détecter de nombreuses anomalies cardiaques telles que les arythmies, qui correspondent à des battements irréguliers du cœur comme la fibrillation auriculaire ou les extrasystoles ; les blocs de conduction, qui se traduisent par une interruption ou un ralentissement du signal électrique dans certaines parties du cœur ; les signes d'ischémie ou d'infarctus, reflétant un défaut d'oxygénation du myocarde, généralement visibles à travers des altérations du segment ST ou de l'onde T ou encore l'hypertrophie cardiaque, qui correspond à une augmentation de la masse musculaire du cœur et peut modifier l'amplitude ou la durée des ondes électrocardiographiques. Grâce à sa simplicité d'utilisation, sa rapidité et sa valeur diagnostique, l'ECG constitue une méthode de première ligne pour l'évaluation cardiaque.[10]

1.6.4 Types d'ECG et dispositifs d'acquisition

Il existe différents types d'ECG selon la durée de l'enregistrement et le contexte clinique :

- **ECG standard de repos** : réalisé sur une courte durée, en position allongée.
- **ECG d'effort** : effectué lors d'un exercice physique pour détecter des anomalies induites.
- **Holter ECG** : enregistrement continu sur 24 à 48 heures à l'aide d'un dispositif portable.
- **Systèmes ECG connectés** : dispositifs portables modernes intégrés dans des montres, patchs ou systèmes embarqués, permettant une surveillance à distance.

Ces différents types d'ECG répondent à des besoins médicaux variés, allant du diagnostic ponctuel en cabinet à la télésurveillance prolongée dans un cadre ambulatoire ou à domicile.[11]

1.6.5 Applications médicales de l'ECG

Avec l'essor des systèmes embarqués et de l'Internet des objets (IoT), l'ECG a franchi une nouvelle étape. Il est désormais possible d'intégrer des capteurs ECG miniaturisés à des plateformes embarquées (comme l'ESP32 ou les microcontrôleurs STM32), connectées à des serveurs locaux ou à des applications mobiles. Cette intégration rend possible une surveillance en temps réel, mobile et intelligente, même en dehors du cadre hospitalier.[12]

L'ECG est un outil central dans de nombreuses applications cliniques :

- Diagnostic des pathologies cardiaques : infarctus du myocarde, fibrillation auriculaire, tachycardies, etc.
- Surveillance continue des patients hospitalisés ou à domicile, notamment en cas de pathologies chroniques.
- Évaluation de traitements médicamenteux ou post-chirurgicaux.
- Intégration dans les systèmes de santé connectés pour la télésurveillance, la prévention et l'alerte précoce.



Figure 1.7 : Utilisation du signal ECG par diffèrent acteurs médicaux

1.7 Conclusion :

Ce premier chapitre a permis d'introduire les bases physiologiques du fonctionnement du cœur et les principes fondamentaux de l'électrocardiogramme (ECG). Nous avons vu que l'activité électrique du cœur, générée par un système de conduction spécialisé, est essentielle à la synchronisation des contractions cardiaques, assurant ainsi une circulation sanguine efficace dans l'organisme. L'ECG, en tant qu'outil de diagnostic, permet de visualiser cette activité électrique sous forme d'ondes caractéristiques, chacune correspondant à une étape précise du cycle cardiaque. Plusieurs types d'enregistrements ECG existent, allant du modèle standard de repos aux dispositifs portables connectés, offrant des possibilités de surveillance continue et à distance.

Cette compréhension du cœur et de l'ECG constitue une base essentielle pour appréhender les enjeux du monitoring médical moderne, et prépare ainsi le terrain pour l'étude des technologies numériques et connectées dans le domaine de la santé, qui feront l'objet du chapitre suivant.

Chapitre 2: Internet des Objets (IoT) et son application dans le domaine médical

Chapitre 2 : Internet des objets (IoT) et son application dans le domaine médical

2.1 Introduction

L'Internet des Objets (IoT — Internet of Things) désigne un ensemble de dispositifs physiques connectés à Internet, capables de collecter, d'échanger et parfois d'analyser des données en temps réel. Ces objets intelligents, dotés de capteurs, d'actionneurs et de capacités de communication, jouent un rôle croissant dans de nombreux domaines tels que l'industrie, la domotique, la logistique ou encore la santé. Dans une architecture IoT typique, les objets connectés recueillent des données physiques (température, mouvement, pression, fréquence cardiaque, etc.) qu'ils transmettent à un serveur ou à une plateforme de traitement via des protocoles de communication sans fil (Wi-Fi, Bluetooth, LoRa…). Ces données sont ensuite exploitées par des applications distantes à des fins de visualisation, d'automatisation ou d'aide à la décision. L'IoT présente plusieurs avantages notables : il permet une surveillance en continu, une meilleure réactivité face aux événements, une automatisation des tâches, et une optimisation des ressources. Son adoption rapide est notamment rendue possible par la miniaturisation des capteurs, la baisse des coûts des microcontrôleurs, et l'essor des solutions logicielles légères et accessibles.[13]

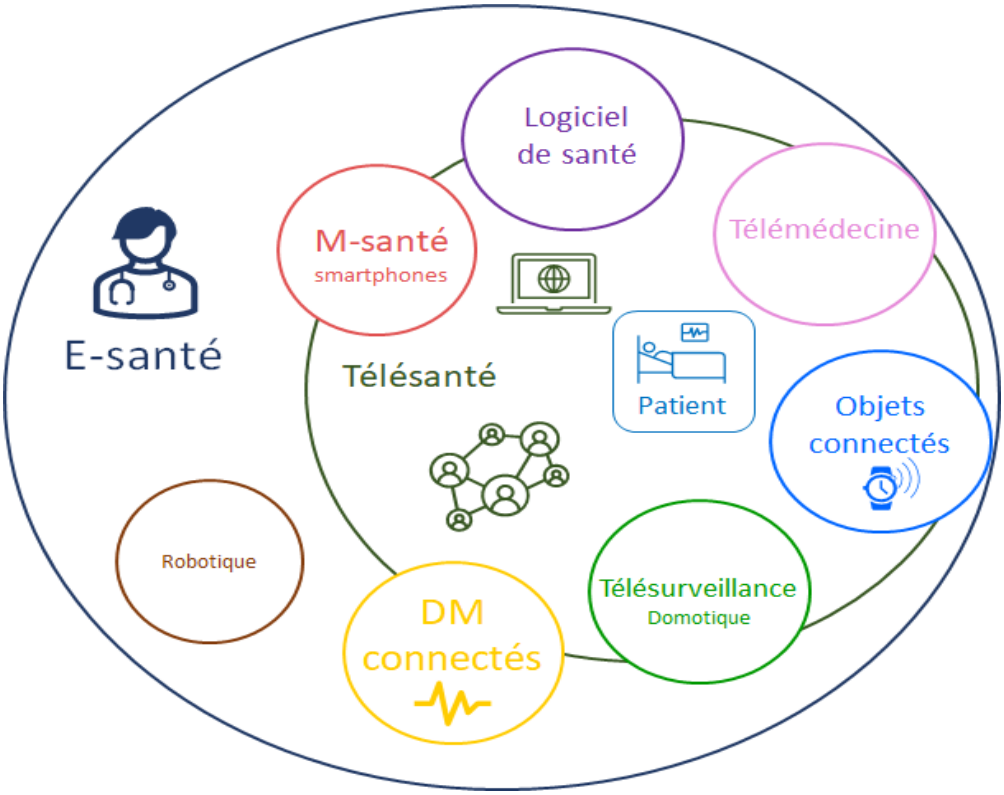


Figure 2.1 : Vue d'ensemble d'un système de santé connecté

2.2 Internet des Objets Médicaux (IoMT)

L'Internet des Objets Médicaux, ou IoMT (Internet of Medical Things), désigne l'ensemble des objets connectés spécifiquement conçus pour des usages cliniques, hospitaliers ou de santé personnelle. Cette déclinaison de l'IoT a pour objectif principal d'améliorer la qualité des soins en assurant une surveillance continue, automatisée et à distance des paramètres vitaux des patients.

2.2.1 Domaines d'application du IoMT

L'IoMT s'applique à de nombreux contextes médicaux :

- **Surveillance à distance des patients :** les dispositifs portables ou implantables permettent le suivi en temps réel du rythme cardiaque, de la tension artérielle, de la glycémie, etc., particulièrement utile pour les maladies chroniques comme le diabète ou l'insuffisance cardiaque.
- **Optimisation des opérations hospitalières :** l'IoMT permet de localiser les équipements, suivre les stocks de médicaments, et automatiser certains processus pour accroître l'efficacité.
- **Amélioration des soins personnalisés :** les données collectées aident à ajuster les traitements selon les besoins réels du patient, avec une prise de décision clinique plus rapide et précise.



Figure 2.2 : Ecosystème médical interconnecté

2.2.2 Exemples de dispositifs IoMT :

L'IoMT regroupe une grande diversité de dispositifs intelligents, adaptés à différents besoins médicaux, comme présenté dans le tableau 2.1.

Tableau 2.1 : Exemples de dispositifs IoMT et leurs fonctions principales.

Type de dispositif	Fonction principale
Montres connectées	Suivi de l'activité physique, du sommeil, de la fréquence cardiaque
Patchs intelligents	Mesure continue de la glycémie, de la température ou de l'ECG
Systèmes de téléassistance	Aide à domicile pour les personnes âgées ou dépendantes
Distributeurs intelligents de médicaments	Rappel et suivi des prises de médicaments
Robots médicaux	Assistance à la mobilité, chirurgie assistée
Réalité virtuelle médicale	Simulation de procédures, formation du personnel soignant

2.2.3 Bénéfices du IoMT :

L'adoption de l'IoMT présente de nombreux avantages, tant pour les professionnels de santé que pour les patients. En facilitant la surveillance continue des signes vitaux, l'échange instantané d'informations médicales et l'analyse intelligente des données, l'IoMT contribue à améliorer la qualité et l'efficacité des soins tout en optimisant les ressources disponibles. Parmi les principaux bénéfices, on peut citer :

- Meilleure qualité des soins grâce à un suivi plus précis et continu.
- Réduction des coûts de santé via la prévention, la détection précoce et la diminution des hospitalisations.
- Personnalisation des traitements rendue possible par l'analyse fine des données.
- Amélioration de l'accès aux soins pour les zones rurales ou les patients à mobilité réduite.[14]

2.3 Les serveurs dans l'architecture IoT

Dans une architecture IoT, les serveurs jouent un rôle fondamental dans la gestion, le traitement et le stockage des données issues des objets connectés. Ils assurent l'interface entre les capteurs physiques déployés sur le terrain et les applications clientes (interfaces web, applications mobiles, plateformes cloud).

2.3.1 Rôle des serveurs dans l'IoT

Au cœur de l'infrastructure de l'Internet des Objets (IoT), le serveur occupe une place centrale dans la gestion, le traitement et la valorisation des données issues des objets connectés. Il assure tout d'abord la réception des informations transmises par les capteurs, en s'appuyant sur des protocoles de communication courants tels que HTTP, MQTT ou CoAP. Une fois les données reçues, le serveur peut les traiter ou les filtrer**,** en appliquant par exemple des règles de validation ou des algorithmes simples comme la moyenne glissante, afin d'en améliorer la qualité.

Les données sont ensuite stockées dans des bases de données, qu'elles soient locales ou distantes**,** généralement en temps réel pour permettre un accès rapide. Le serveur met également à disposition une API (Application Programming Interface), permettant aux autres applications, services ou utilisateurs autorisés d'accéder aux données de manière sécurisée. Enfin, la sécurité des échanges constitue une fonction essentielle du serveur, impliquant des mécanismes d'authentification, de chiffrement, ainsi que de gestion des autorisations d'accès, afin de garantir la confidentialité et l'intégrité des données médicales.[15]

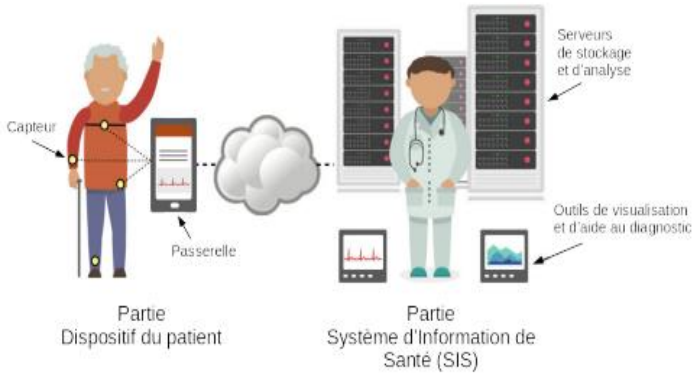


Figure 2.3 : Fonctionnement d'un serveur dans un système IoT médical

En fonction des contraintes du système (coût, latence, autonomie énergétique, volume de données), plusieurs types de serveurs peuvent être utilisés :

- **Serveurs locaux embarqués** : hébergés sur une carte comme le Raspberry Pi, adaptés aux systèmes autonomes ou isolés (edge computing).
- **Serveurs cloud** : hébergés sur des plateformes en ligne (AWS, Azure, Firebase…), idéals pour les projets à grande échelle ou les applications nécessitant un accès distant global.
- **Serveurs hybrides** : combinant un traitement local (edge) pour les tâches immédiates et une synchronisation périodique avec un serveur cloud pour l'analyse à long terme.

2.3.2 Serveur Flask : un microframework léger adapté à l'IoT

Dans le contexte des systèmes IoT embarqués, Flask s'impose comme un outil de choix pour le développement de serveurs légers et efficaces. Il s'agit d'un microframework web développé en Python, reconnu pour sa simplicité, sa flexibilité et sa légèreté, ce qui le rend particulièrement adapté aux environnements à ressources limitées, comme les Raspberry Pi. Grâce à son architecture minimaliste, Flask permet de concevoir rapidement des API RESTful pour la réception, le traitement et la diffusion des données, tout en offrant aux développeurs la liberté d'ajouter uniquement les extensions nécessaires (authentification, gestion de base de données, interface web, etc.).

Dans une architecture IoT, Flask est couramment utilisé pour créer des points de terminaison HTTP permettant aux capteurs de communiquer via des requêtes GET ou POST, pour interfacer une base de données locale (comme SQLite), ou encore pour fournir une interface web accessible à distance permettant la visualisation en temps réel des données collectées. Comme illustré dans la figure 2.4, il peut également intégrer des logiques métiers simples, localement, sans dépendre d'un service cloud. Cette souplesse fait de Flask un outil central pour le développement de serveurs embarqués, où efficacité, robustesse et clarté du code sont essentiels. [16]

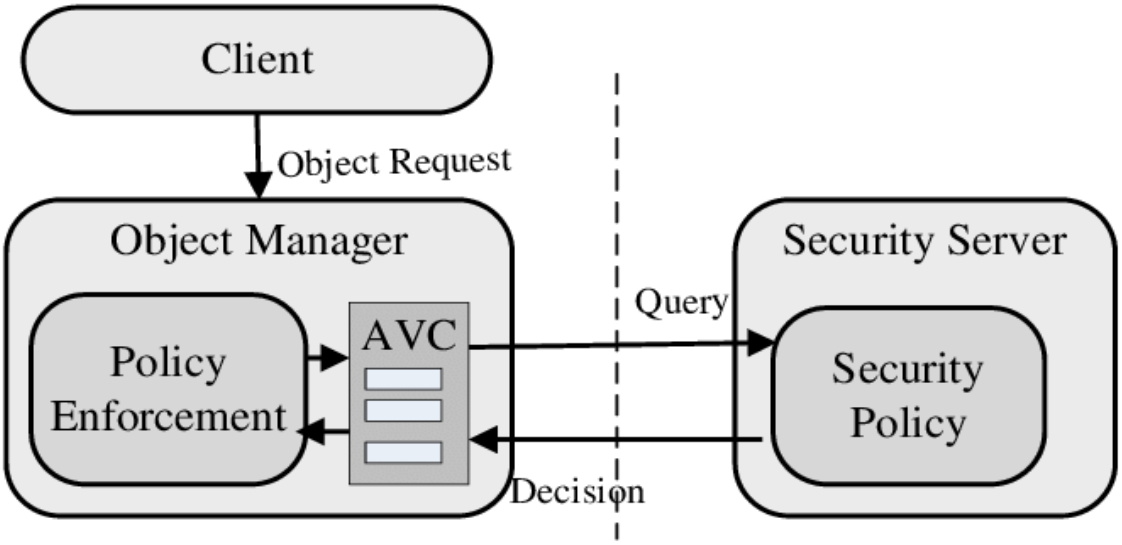


Figure 2.4 : Architecture fonctionnelle d'un serveur Flask dans un système IoT

2.4 Les bases de données dans les systèmes IoT

Dans une architecture IoT, la base de données joue un rôle fondamental : elle assure la persistance des données envoyées par les objets connectés. Autrement dit, elle permet de stocker de manière structurée l'historique des mesures collectées (température, fréquence cardiaque, humidité, etc.) afin de pouvoir les exploiter, les visualiser ou les analyser a posteriori.

2.4.1 Rôle et importance des bases de données en IoT

Les objets connectés génèrent en continu un flux de données. Pour éviter leur perte et garantir la traçabilité, ces données doivent être conservées durablement dans une base. Ce stockage permet de :

- **Retrouver l'historique des mesures** (utile pour les comparaisons, les alertes, ou les diagnostics médicaux par exemple).
- **Alimenter des interfaces utilisateurs** (dashboards, applications mobiles…) en données actualisées.
- **Servir de support à l'analyse de données**, notamment dans le cadre de l'intelligence artificielle ou du machine learning.
- **Partager ou synchroniser les données** entre plusieurs systèmes distants.

Dans les architectures embarquées ou locales (comme sur un Raspberry Pi), on privilégie des solutions légères, peu consommatrices en ressources, et faciles à mettre en œuvre.[17]

2.4.2 SQLite : une base de données légère adaptée aux systèmes embarqués

SQLite est une **base de données relationnelle** embarquée, écrite en C, qui stocke les données dans un fichier unique. Contrairement à des systèmes comme MySQL ou PostgreSQL, elle ne nécessite ni serveur ni processus de gestion indépendant, ce qui la rend idéale pour les environnements à faibles ressources.

Avantages de SQLite dans un contexte IoT :

- **Installation simplifiée** : aucun service à lancer, simplement une bibliothèque à intégrer.
- **Légèreté** : très faible empreinte mémoire et CPU.
- **Compatibilité avec Python** : elle s'intègre parfaitement à Flask grâce au module sqlite3.
- **Robustesse** : largement testée et utilisée, elle convient même à des applications industrielles.

Par conséquent, SQLite constitue une solution pertinente pour le stockage local des données biomédicales dans un système IoT, assurant un bon compromis entre simplicité, fiabilité et efficacité.[18]

2.4.3 Interaction entre serveur et base de données

Dans un système IoT, le serveur joue le rôle d'intermédiaire entre les objets connectés et la base de données. Lorsqu'un capteur envoie une nouvelle mesure (par exemple, la température ambiante ou le rythme cardiaque), cette donnée transite par le serveur avant d'être enregistrée dans la base. Le serveur traite donc deux types d'opérations principales :

- **Insertion des données** : à chaque requête HTTP POST reçue depuis un capteur, le serveur Flask extrait les informations utiles et les insère dans la base SQLite. Cela nécessite une structure de table définie en amont (par exemple : date, type de capteur, valeur mesurée).
- **Lecture des données** : les interfaces utilisateur (page web, application mobile) effectuent des requêtes HTTP GET pour consulter les données. Le serveur interroge alors la base de données, extrait les résultats, puis les renvoie au format JSON.

Cette communication entre le serveur Flask et la base SQLite repose sur des commandes SQL (langage de requête structuré) exécutées via des bibliothèques Python telles que sqlite3 ou des ORM (Object Relational Mapper) comme SQLAlchemy.

L'efficacité de cette interaction illustre dans la figure 2.5, conditionne la fluidité globale du système, des index bien pensés et des requêtes optimisées permettent de maintenir de bonnes performances, même sur une plateforme légère comme le Raspberry Pi, fréquemment utilisée comme système embarqué dans les applications IoT. [19]

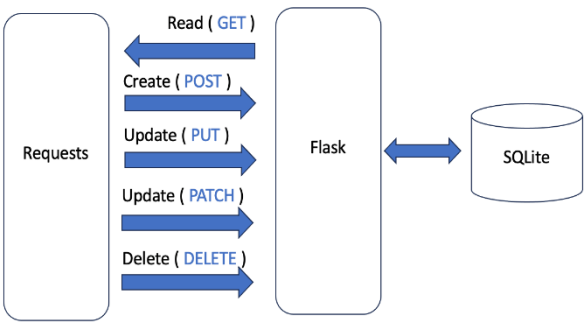


Figure 2.5 Structure montrant l'interaction entre le serveur et la base de données

2.5 Les interfaces utilisateurs dans l'IoT médical

En Iot, les interfaces utilisateurs jouent un rôle fondamental : elles permettent la consultation, l'interprétation et parfois l'interaction avec les données physiologiques collectées par les capteurs. Ces interfaces peuvent prendre la forme d'une page web ou d'une application mobile, selon les besoins des utilisateurs finaux (médecins, patients, techniciens).

2.5.1 Les interfaces WEB

Les interfaces web sont particulièrement adaptées pour l'affichage en temps réel des paramètres vitaux. Elles permettent d'accéder aux données depuis n'importe quel appareil connecté à un réseau, sans nécessiter d'installation spécifique.

Technologies utilisées des interfaces WEB : HTML, CSS, JavaScript

La structure, le style et la dynamique d'une page web sont assurés par l'utilisation conjointe de trois langages fondamentaux :

- **HTML (HyperText Markup Language)** : structure le contenu de la page (titres, tableaux, boutons, etc.).
- **CSS (Cascading Style Sheets)** : définit l'apparence (mise en page, couleurs, polices).
- **JavaScript** : rend la page interactive et permet l'actualisation des données sans rechargement complet.

Ces technologies sont légères, compatibles avec tous les navigateurs modernes, et particulièrement bien adaptées aux projets embarqués et IoT. [20]

2.5.2 Interaction avec le serveur Flask

L'interface web communique avec le serveur Flask à l'aide de requêtes HTTP, souvent via la méthode fetch() ou des appels AJAX. Ces requêtes interrogent les routes définies dans le serveur Flask, qui renvoie les données au format JSON. Ainsi, les informations reçues peuvent être directement injectées dans l'interface utilisateur.

Cette méthode permet une mise à jour dynamique des valeurs sans recharger toute la page, tout en maintenant une faible charge réseau et une bonne réactivité.

2.5.3 Visualisation des données en temps réel avec Chart.js

Pour représenter graphiquement l'évolution des paramètres vitaux (comme l'ECG, la température ou la fréquence cardiaque), on utilise des bibliothèques JavaScript spécialisées comme Chart.js**.** Chart.js permet de créer facilement des courbes, des graphiques en barres ou en radar, avec un rendu fluide et personnalisable comme illustré dans la figure 2.6. Les données collectées depuis le serveur Flask sont injectées en temps réel dans ces graphes, offrant à l'utilisateur une vision continue de l'état de santé du patient.

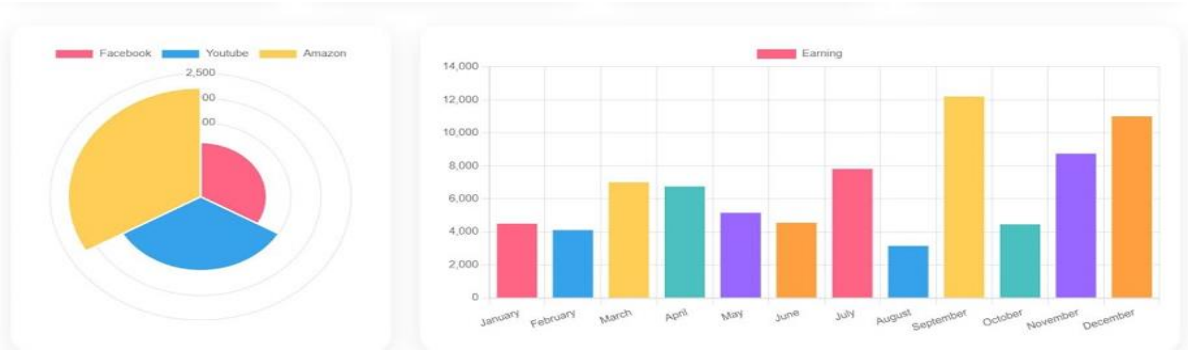


Figure 2.6 : Exemple d'interface avec Chart.js

2.5.4 Application mobile

En parallèle de l'interface web, les applications mobiles offrent une portabilité et une accessibilité accrues. Dans le domaine médical, elles facilitent le suivi des patients à distance, en permettant aux professionnels de santé d'accéder aux données à tout moment.

2.5.5 Importance de la mobilité dans le suivi médical

Les solutions mobiles jouent un rôle central dans les systèmes de santé modernes. Elles offrent une consultation nomade des paramètres vitaux, permettant aux professionnels de santé de suivre l'état du patient en temps réel, quel que soit leur emplacement. Elles facilitent également l'envoi de notifications instantanées en cas de dépassement de seuils critiques, assurant une intervention rapide. De plus, elles permettent aux utilisateurs un accès personnalisé et sécurisé à leur historique médical, favorisant ainsi un meilleur suivi à long terme. Ces fonctionnalités contribuent à améliorer significativement la réactivité des soignants et la sécurité des patients.[21]

2.5.6 Présentation de Flutter

Flutter est un framework open-source développé par Google, utilisé pour créer des applications mobiles multiplateformes (Android, iOS) à partir d'un seul code source en langage Dart.

Il est particulièrement adapté aux projets IoT pour plusieurs raisons :

- Temps de développement réduit grâce au hot reload.
- Interface moderne et fluide.
- Compatibilité avec les requêtes HTTP, WebSockets, JSON, etc.

2.5.7 Architecture d'une application mobile Flutter pour l'IoT médical

Les applications mobiles développées avec Flutter dans le domaine de l'Internet des Objets médicaux (IoMT) reposent sur une architecture modulaire, répartie en plusieurs écrans (ou vues) correspondant chacun à une fonctionnalité précise. Il peut s'agir, par exemple, d'un tableau de bord de suivi, d'une page de visualisation de données physiologiques comme l'ECG, ou encore d'un écran d'historique des mesures.
Chaque écran interagit avec un serveur distant (souvent un serveur Flask ou équivalent) à l'aide de requêtes HTTP (GET/POST). Les données récupérées sont ensuite affichées à l'aide de widgets réactifs comme ListView, Card, ou encore via des bibliothèques de visualisation comme fl_chart, permettant une présentation graphique en temps réel des paramètres vitaux.
Cette approche permet de concevoir des interfaces mobiles performantes, adaptatives et facilement maintenables, tout en assurant une bonne séparation entre la logique métier, la récupération de données, et l'interface utilisateur.

2.5.8 Synchronisation avec le serveur Flask

Dans les systèmes IoT médicaux, l'application mobile est souvent configurée pour interroger régulièrement le serveur de données (par exemple un serveur Flask) afin d'assurer un affichage en temps réel des mesures physiologiques. Cette interrogation périodique peut varier selon le type de données à surveiller : une fréquence rapide pour des signaux continus comme l'ECG (toutes les 500 millisecondes), et une fréquence plus espacée (toutes les quelques secondes) pour d'autres capteurs comme la température, l'oxygénation ou la qualité de l'air.
Ce mécanisme de synchronisation permet de maintenir une cohérence temporelle entre les données affichées sur l'interface mobile, celles disponibles via le serveur web, et celles stockées dans la base de données. Il constitue un élément clé pour garantir la fiabilité du suivi médical en temps réel.

2.6 Protocoles de communication : HTTP et Wi-Fi

Dans les systèmes IoT, les protocoles de communication jouent un rôle crucial en assurant l'échange de données entre les objets connectés, les serveurs et les interfaces utilisateurs. Deux protocoles essentiels se distinguent dans les architectures légères et accessibles : **le Wi-Fi** pour la connexion réseau sans fil, et **le protocole HTTP** pour le transfert de données via Internet.

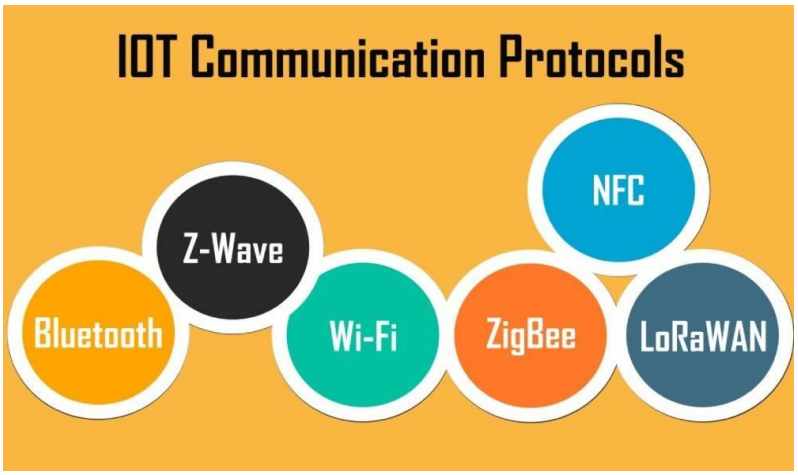


Figure 2.7 : Les différents protocole de communications en IOT

2.6.1 Le Wi-Fi en IOT

Le Wi-Fi (Wireless Fidelity) est un protocole de communication sans fil basé sur la norme IEEE 802.11. Il est largement utilisé dans les environnements domestiques, industriels et médicaux pour connecter les objets à un réseau local (LAN) ou à Internet. Dans un système IoT, le Wi-Fi permet à un microcontrôleur, comme un ESP32 ou un Raspberry Pi, de se connecter directement à un point d'accès (routeur) pour émettre ou recevoir des données. L'un des principaux avantages du Wi-Fi en IoT réside dans son débit élevé, ce qui le rend particulièrement adapté aux transmissions fréquentes de données, comme les signaux physiologiques. De plus, cette technologie bénéficie d'une infrastructure déjà très répandue, ce qui évite d'avoir recours à du matériel supplémentaire. Elle est également compatible avec une grande variété de protocoles de communication tels que HTTP, MQTT ou WebSocket. Toutefois, le Wi-Fi présente certaines limites, notamment une consommation énergétique moyenne à élever, ce qui le rend moins adapté aux dispositifs fonctionnant sur batterie, ainsi qu'une portée restreinte à quelques dizaines de mètres en moyenne.[22]

2.6.2 Le protocole HTTP en IoT :

Le protocole HTTP est l'un des plus répandus en raison de sa simplicité de mise en œuvre et de sa parfaite compatibilité avec les technologies web existantes. Il s'intègre facilement aux architectures RESTful et convient bien aux microcontrôleurs. Cependant, son utilisation dans des contextes contraints présente également des inconvénients. En effet, HTTP repose sur une surcouche textuelle qui augmente la taille des messages, ne permet pas une communication en temps réel par défaut, contrairement à des alternatives comme MQTT ou WebSocket, et consomme plus d'énergie, chaque requête nécessitant l'ouverture et la fermeture d'une connexion TCP. Le tableau ci-dessous 2.2 résume de manière synthétique les principaux avantages et inconvénients de l'utilisation du protocole HTTP dans le cadre de L'IoT.[23]

Tableau 2.2 : Comparaison des avantages et inconvénients du protocole HTTP dans un contexte IoT.

Avantages	Inconvénients
Simplicité de mise en œuvre, notamment dans les microcontrôleurs	Surcouche textuelle, augmentant la taille des messages
Interopérabilité avec les technologies web modernes	Pas de communication en temps réel par défaut (contrairement à MQTT/WebSocket)
Compatible avec les architectures RESTful largement utilisées en IoT	Moins économe en énergie, car chaque requête nécessite une ouverture/fermeture de connexion TCP

2.7 Conclusion :

Dans ce chapitre, nous avons exploré les fondations techniques indispensables à la conception d'un système de santé connecté. Le rôle des serveurs IoT a été mis en évidence, notamment dans la réception, le traitement et le stockage sécurisé des données issues des capteurs, à l'aide de protocoles comme HTTP et de bases de données telles que SQLite.

Nous avons également souligné l'importance des interfaces utilisateurs, qu'elles soient web ou mobiles, dans la visualisation claire et intuitive des informations médicales. L'ensemble de ces éléments constitue l'infrastructure matérielle et logicielle sur laquelle repose un système de télésurveillance efficace, ouvrant la voie à l'intégration concrète des capteurs biomédicaux et des logiciels utilise dans le chapitre suivant.

Chapitre 3 : Conception et réalisation de notre système de sante connecté :

Chapitre 3 : Conception et réalisation de notre système de sante connectée

3.1 Introduction :

L'objectif de ce chapitre est de concevoir une solution complète, à la fois matérielle et logicielle, qui permette de collecter, traiter, visualiser et stocker automatiquement des données biomédicales critiques, le tout dans un environnement sécurisé et localement contrôlé.

Ce chapitre décrit donc en détail les deux volets principaux du système :

- La partie matérielle, avec le choix du microcontrôleur ESP32 et des différents capteurs biomédicaux (ECG, température/humidité, SpO2, qualité de l'air), leur interconnexion et leur rôle dans la chaîne de mesure.
- La partie logicielle, avec la mise en place du serveur Flask, la base de données SQLite, l'interface web, et l'application mobile Flutter.

Ce projet a été inspiré par une discussion avec un médecin généraliste, qui m'a fait part du besoin d'outils simples et fiables pour le suivi à distance des patients. Elle a souligné l'intérêt d'un système capable de surveiller en temps réel les paramètres vitaux essentiels, tout en assurant une traçabilité rigoureuse et sécurisée des données médicales. L'enregistrement automatique et sécurisé de ces données permettrait notamment de mieux interpréter l'évolution d'un état clinique, de documenter précisément un décès, ou de détecter une anomalie à temps chez un patient vulnérable.

Cette discussion a été le point de départ de ce projet, et a orienté plusieurs choix techniques et fonctionnels, à commencer par l'ergonomie du système, qui repose sur une interface web claire et une application mobile intuitive, à la fois pour les soignants et les proches aidants. Un effort particulier a été porté sur la visualisation simple des données, l'accessibilité en temps réel, et l'automatisation du stockage.

D'un point de vue technique, notre système repose sur une architecture modulaire, déployée en réseau local pour des raisons de sécurité et de contrôle total des données. En effet, contrairement aux plateformes cloud dont les données sont souvent hébergées à l'extérieur, notre solution s'appuie sur un serveur personnel (Raspberry Pi 5) local, que j'ai moi-même configuré, ce qui garantit une maîtrise complète de l'environnement logiciel et réseau. Cela rend le système non seulement plus robuste face aux intrusions, mais également plus respectueux de la confidentialité des données médicales.

3.2 Architecture fonctionnelle du système :

Afin de mieux illustrer l'architecture globale de notre système de santé connecté, nous présentons dans la figure ci-dessous un organigramme 3.1 fonctionnel décrivant les interactions entre les différents composants matériels et logiciels.

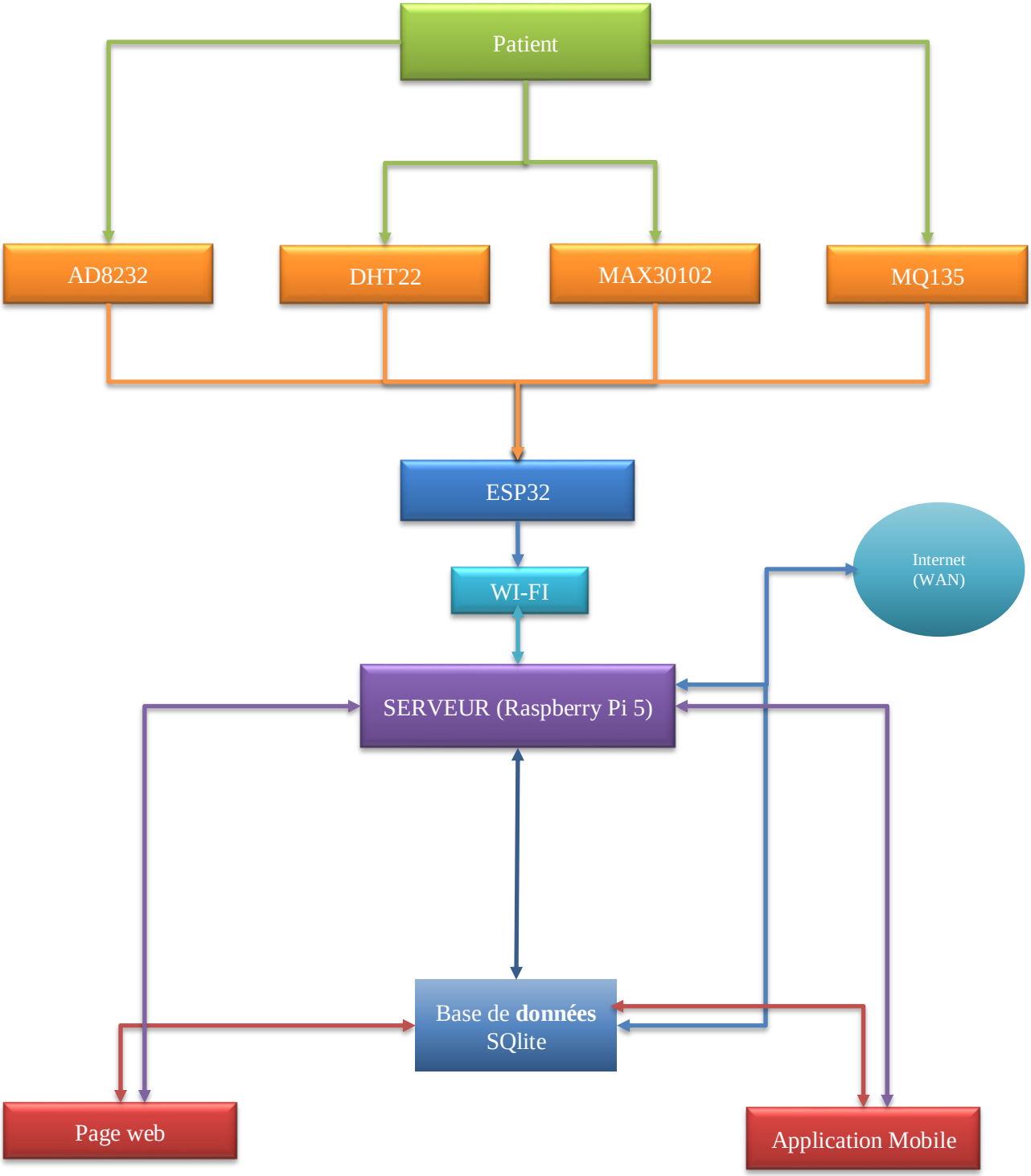


Figure 3.1 : Organigramme fonctionnel du système de santé connecté.

3.3 Partie matérielle :

La partie matérielle du système repose sur une architecture légère et adaptée aux systèmes embarqués, avec l'ESP32 comme microcontrôleur central. Il est connecté à plusieurs capteurs biomédicaux (ECG AD8232, DHT22, MAX30102, MQ-135) permettant de mesurer en temps réel les principaux paramètres vitaux. Cette section détaille les caractéristiques de ces composants, leur intégration, ainsi que les tests de validation effectués, en veillant à concilier fiabilité médicale, faible consommation et faisabilité académique.

3.3.1 Microcontrôleurs et micro-ordinateur embarqués

a) Carte de développement ESP32

L'ESP32 reçoit le signal ECG analogique issu du capteur AD8232 sur l'une de ses entrées ADC. Une fois ce signal converti en données numériques, il applique un traitement préliminaire, tel qu'un filtrage logiciel léger ou une normalisation, avant de l'envoyer via HTTP POST vers le serveur Flask hébergé sur le Raspberry Pi 5. Ce microcontrôleur agit donc à la fois comme acquisiteur de données**,** préprocesseur**,** et émetteur sans fil**.** Il permet un fonctionnement autonome du capteur sans nécessité de passer par un ordinateur, ce qui en fait un élément fondamental pour les applications portables ou mobiles. Il convient également de prévoir une alimentation stable et adaptée, souvent assurée via une batterie ou une source externe, pour garantir un fonctionnement continu.[24]



Figure 3.2 : Carte de développement ESP32.

Caractéristiques techniques principales :

- Microprocesseur dual-core Xtensa 32 bits cadencé jusqu'à 240 MHz
- Mémoire : jusqu'à 520 Ko de SRAM et 4 Mo de flash externe
- Convertisseur analogique-numérique (ADC) 12 bits intégré
- Connectivité Wi-Fi 802.11 b/g/n et Bluetooth v4.2 (BLE)
- Nombreuses interfaces E/S : GPIO, UART, SPI, I2C, PWM, etc.
- Alimentation : 3,3V (via régulateur intégré sur les modules standards)
- Format compact et adaptable (modules DevKitC, WROOM32, etc.)

b) Le Raspberry Pi 5

Le Raspberry Pi 5 illustré dans la figure si dessous 3.3 constitue le cœur du système de santé connecté que nous avons conçu. Il est en charge du traitement, du stockage et de l'affichage des données, il a aussi été choisi pour soulager l'ESP32 des tâches gourmandes en ressources, notamment celles liées à l'hébergement du serveur, à la gestion de la base de données et à l'affichage en temps réel via une interface web. Grâce à sa puissance de calcul élevée, il permet d'assurer un fonctionnement fluide et fiable de l'ensemble du système. Il est toutefois essentiel de lui fournir une alimentation stable et suffisante de 5V / 5A, sans quoi des redémarrages ou dysfonctionnements pourraient survenir, notamment lors de charges élevées. La carte que nous avons utilisée dans ce projet est équipée de 16 Go de RAM, ce qui offre une marge confortable pour exécuter plusieurs services simultanément sans ralentissement.

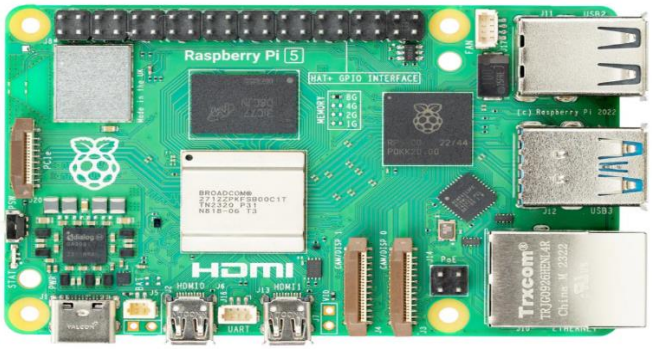


Figure 3.3 : Carte Raspberry Pi 5.

Caractéristiques techniques principales :

- Processeur : Quad-core ARM Cortex-A76 cadencé à 2.4 GHz, offrant une puissance de traitement adaptée aux tâches embarquées complexes
- Mémoire vive : 16 Go LPDDR4X, garantissant une fluidité dans l'exécution simultanée de plusieurs services
- Stockage : possibilité d'utiliser une carte microSD (dans notre cas, une carte de 128 Go a été utilisée) ou un SSD via le port PCIe pour des performances de lecture/écriture améliorées
- Connectivité sans fil : Wi-Fi 802.11ac et Bluetooth 5.0 intégrés, permettant une connexion facile aux autres éléments du système
- Compatibilité avec les outils de développement
- Port GPIO à 40 broches : idéal pour la connexion de composants ou capteurs supplémentaires

Rôle fonctionnel dans notre système de santé connecté :

Le Raspberry Pi 5 héberge une API REST développée avec le micro-framework Flask en Python, chargée de recevoir les données ECG transmises par l'ESP32 via des requêtes HTTP POST. Ces données sont ensuite automatiquement enregistrées dans une base de données SQLite, choisie pour sa légèreté, sa simplicité d'intégration et son efficacité dans un environnement embarqué.

En parallèle, le Raspberry Pi héberge également une interface web locale, accessible depuis un navigateur, permettant de visualiser les signaux ECG en temps réel. Conçue en HTML, CSS et JavaScript, cette interface s'appuie sur la bibliothèque Chart.js pour l'affichage dynamique des courbes électrocardiographiques, offrant ainsi une lecture claire et immédiate des signaux vitaux.

Avantages liés à l'utilisation du Raspberry Pi 5 :

Le Raspberry Pi 5 s'est imposé comme une solution performante, accessible et évolutive pour notre système de santé connecté, offrant une base fiable pour le prototypage et l'intégration de futures extensions. Le tableau suivant en résume les principaux avantages et inconvénients dans ce contexe

Tableau 3.1 : Avantages et inconvénients de la Raspberry Pi 5

Avantages	Inconvénients
Cœur central du système, il décharge l'ESP32 des tâches critiques	Peut nécessiter un refroidissement actif (ventilateur) en cas de forte charge

Format compact et faible consommation, idéal pour les systèmes embarqués portables	Moins adapté aux architectures distribuées avec de nombreux utilisateurs simultanés
Grande compatibilité logicielle (Python, Flask, SQLite, etc.)	Dépendance à une alimentation stable : une coupure peut interrompre les services critiques du système

3.3.2Les capteurs

a) Module ECG AD8232 :

Le capteur **AD8232** montré dans la figure 3.4 est un module spécialisé dans la mesure des signaux électrocardiographiques (ECG). Il est conçu pour extraire, amplifier et filtrer les signaux électriques du cœur, même dans des environnements présentant un fort bruit électromagnétique, rendant les données fiables et exploitables par des microcontrôleurs tels que l'ESP32. Ce capteur est un composant clé dans de nombreux projets de télésanté grâce à sa simplicité d'intégration et sa précision.

Caractéristiques techniques principales :

- Alimentation : 3,3V ou 5V, offrant une flexibilité pour les systèmes à basse consommation
- Sortie : signal analogique proportionnel à l'activité électrique du cœur
- Filtres intégrés : suppression du bruit haute fréquence et filtrage passe-bande, garantissant un signal propre
- Dimensions compactes : facile à intégrer dans des boîtiers portables ou des wearables

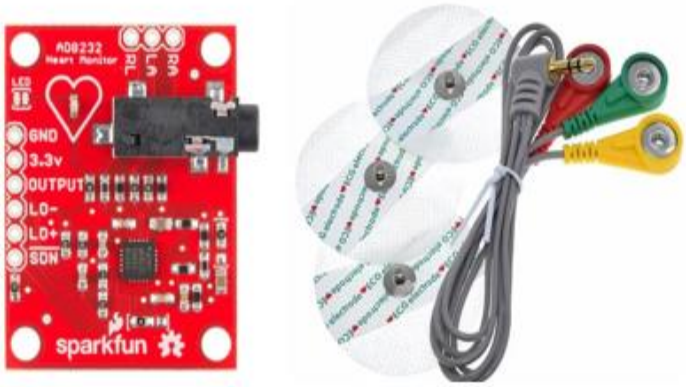


Figure 3.4 : Capteur ECG AD8232.

Fonctionnement : Le capteur repose sur un système de trois électrodes cutanées destinées à capter les signaux électriques émis par l'activité cardiaque. Ces électrodes peuvent être positionnées de deux manières différentes, selon la configuration choisie ou le confort du patient, comme illustre dans la figure 3.5. Classiquement, on place une électrode de référence (GND) sur la jambe droite, une électrode positive sur le bras droit, et une électrode négative sur le bras gauche. Le module AD8232 amplifie ces signaux faibles, les filtre pour supprimer les parasites et les stabiliser, puis les transmet à un microcontrôleur pour traitement numérique.

Le signal analogique en sortie du capteur est connecté à une entrée analogique de l'ESP32, qui effectue la numérisation à l'aide de son convertisseur analogique-numérique (ADC).

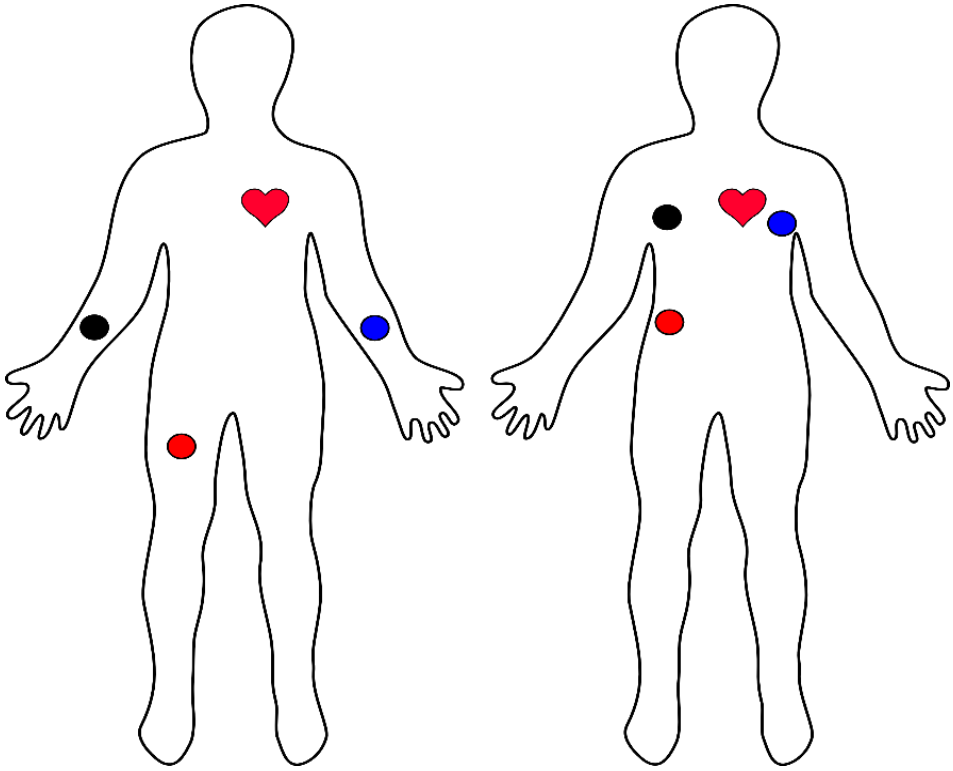


Figure 3.5 : Différents placements des électrodes sur le corps.

Avantages et inconvénients du capteur AD8232 :

Tableau 3.2 : Avantages et inconvénients du capteur AD8232

Avantages	Inconvénients
Haute précision dans la mesure des signaux ECG, adaptée aux applications médicales ou de recherche.	Sensibilité au bruit électromagnétique, nécessitant un blindage ou une filtration supplémentaire.

Faible consommation énergétique (3,3 V ou 5 V), idéal pour les dispositifs portables autonomes.	Dépendance d'un placement précis des électrodes pour des lectures fiables, pouvant poser des défis pratiques.
Filtres intégrés (suppression du bruit haute fréquence et passe-bande), garantissant un signal propre.	Gain ajustable complexe, nécessitant une calibration pour chaque configuration spécifique.

b) Capteur SpO2 et fréquence cardiaque MAX30102

Le capteur MAX30102 montré dans la figure 3.6 est un module spécialisé dans la mesure simultanée de la saturation en oxygène du sang (SpO2) et de la fréquence cardiaque (BPM), jouant un rôle essentiel dans notre système de santé connecté. Conçu par Maxim Integrated, ce capteur repose sur la technologie de la photopléthysmographie (PPG), qui utilise des LED rouges, infrarouges et, dans certaines variantes, vertes pour détecter les variations de volume sanguin dans les capillaires, même en présence de bruit ambiant. Cette capacité le rend particulièrement adapté à une intégration avec un microcontrôleur comme l'ESP32, offrant des données précises pour le monitoring en temps réel des patients.

Caractéristiques techniques :

- **Alimentation :** 1,8 V à 3,3 V, compatible avec les systèmes à faible consommation comme l'ESP32.
- **Sortie :** Données numériques via une interface I2C, facilitant la communication avec le microcontrôleur.
- **Filtres intégrés :** Traitement interne des signaux pour réduire les interférences et améliorer la qualité des mesures.
- **Connexions :** Broches SDA et SCL pour I2C (connectées à GPIO21 et GPIO22 sur l'ESP32), ainsi que VCC, GND, et INT pour les interruptions.

- **Dimensions compactes :** Format réduit (environ 5,6 x 3,3 mm), idéal pour des dispositifs portables ou wearables.

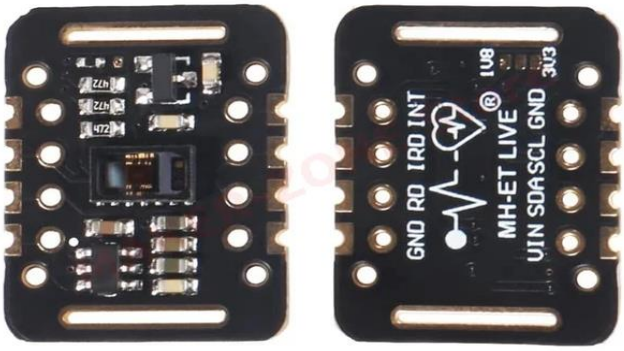


Figure 3.6 : Module du capteur MAX30102 face avant et arrière

Fonctionnement : Le MAX30102 fonctionne en émettant de la lumière à travers la peau (généralement au niveau du doigt ou du lobe de l'oreille) via ses LED. La lumière réfléchie ou transmise est captée par un photo détecteur, permettant de mesurer les variations de l'absorption sanguine. Ces variations sont ensuite analysées pour calculer la SpO2 (en pourcentage, typiquement 95-100 % pour une personne en bonne santé) et le pouls (en battements par minute, généralement 60-100 BPM). Le capteur est positionné sur une zone bien vascularisée, et les données brutes sont traitées par l'ESP32 pour extraire les paramètres vitaux. Nous avons constaté que le placement précis du capteur est crucial pour minimiser les artefacts dus aux mouvements.

c) Le capteur de qualité de l'air MQ-135

Développé par Hanwei Electronics, Le capteur MQ-135 est un module dédié à la détection de la qualité de l'air, jouant un rôle complémentaire dans notre système de santé connecté en surveillant les conditions environnementales susceptibles d'influencer les paramètres vitaux. Nous l'avons intégré à l'ESP32 pour enrichir les données physiologiques avec des informations environnementales, essentielles pour une analyse holistique des patients.

Caractéristiques techniques :

- **Alimentation :** 5 V (avec régulateur si connecté à 3,3 V de l'ESP32).
- **Sortie :** Signal analogique proportionnel à la concentration de gaz (0 à 1023 sur l'ADC de l'ESP32).
- **Sensibilité :** Réponse à des concentrations de gaz typiques de 10 à 1000 ppm, selon la calibration.

- **Dimensions compactes :** Format modulaire (environ 32 x 22 x 27 mm), adapté à une intégration dans des dispositifs portables.

Fonctionnement : Le MQ-135 illustré dans la figure 3.7 fonctionne en détectant les variations de la concentration de gaz dans l'air ambiant grâce à un composant sensible qui réagit chimiquement aux polluants présents. Ces réactions modifient la résistance interne du capteur, générant un signal analogique proportionnel à la concentration de gaz, lu via l'ADC de l'ESP32. Le capteur mesure ainsi la **qualité de l'air**, en particulier les concentrations de gaz comme le dioxyde de carbone (CO_2), l'ammoniac (NH_3) ou le benzène, exprimées en **ppm (parties par million)**.



Figure 3.7 : Capteur de qualité de l'air MQ135

d) Le capteur de température et d'humidité DHT22

Le capteur DHT22 represente dans la figure 3.8 est un module spécialisé dans la mesure simultanée de la température et de l'humidité, constituant un élément clé de notre système de santé connecté pour le monitoring des conditions environnementales et physiologiques. Développé par Aosong, ce capteur utilise un thermistor pour la température et un capteur capacitif pour l'humidité, offrant une précision élevée adaptée aux applications de télésurveillance médicale. Nous l'avons intégré à

l'ESP32 via une interface One-Wire, permettant une collecte de données fiable en temps réel pour évaluer l'environnement du patient ou ses conditions cutanées.

Caractéristiques techniques :

- **Alimentation :** 3,3 V à 5 V, compatible avec les systèmes à faible consommation comme l'ESP32.
- **Sortie :** Données numériques via une interface One-Wire, simplifiant la communication avec le microcontrôleur.
- **Précision :** ±0,5 °C pour la température (plage : 0-50 °C) et ±2-5 % pour l'humidité (plage : 0-100 % RH).
- **Dimensions compactes :** Format encapsulé (environ 15,1 x 25 x 7,7 mm), adapté aux dispositifs portables.



Figure 3.8 : Capteur de température et d'humidité DHT22

Fonctionnement : Le DHT22 mesure la température ambiante ou corporelle via un thermistor et l'humidité relative grâce à un capteur capacitif sensible aux variations d'humidité dans l'air. Les données sont converties en signaux numériques et transmises périodiquement (toutes les 2 secondes minimum) à l'ESP32 via la broche DATA. Nous avons positionné le capteur près du patient pour refléter les conditions locales, en veillant à le protéger des condensations excessives. On observe

que la stabilité des mesures dépend fortement d'une calibration initiale et d'un environnement sans interférences thermiques majeures. La connexion à l'ESP32 est réalisée comme suit : la broche VCC est alimentée par 3.3V, la broche GND est reliée à la masse, et la broche DATA est connectée à GPIO27 avec une résistance pull-up de 4,7 kΩ pour assurer une communication stable via le protocole One-Wire.

3.3.4 Tableau des connexions entre les capteurs et l'ESP32 :

Le tableau suivant 3.3 récapitule les connexions physiques entre les différents capteurs utilisés dans le système (ECG AD8232, capteur de température et d'humidité DHT22, capteur de fréquence cardiaque et SpO_2 MAX30102, capteur de qualité de l'air MQ-135) et les broches de l'ESP32.

Tableau 3.3 : Connexions entre les capteurs et l'ESP32.

Capteur	Broche capteur	Fonction	Broche ESP32
AD8232	OUTPUT	Signal ECG (analogique)	GPIO 34
	Lo+	Détection de déconnexion électrode positive	GPIO 32
	Lo-	Détection de déconnexion électrode négative	GPIO 33
	3.3V	Alimentation	3.3V
	GND	Masse	GND
DHT22	DATA	Donnée numérique	GPIO 27
	VCC	Alimentation	3.3V
	GND	Masse	GND
MAX30102	SDA	Donnée I2C	GPIO 21
	SCL	Horloge I2C	GPIO 22
	VIN	Alimentation	3.3V
	GND	Masse	GND
MQ-135	AOUT	Sortie analogique	GPIO 35
	VCC	Alimentation	3.3V
	GND	Masse	GND

3.3.5Conversion des valeurs brutes ECG en unités physiques exploitables

Lors de l'acquisition d'un signal électrocardiographique (ECG) à l'aide du capteur AD8232 connecté à un ESP32, les données recueillies sont des valeurs numériques brutes issues du convertisseur analogique-numérique (ADC) de l'ESP32. Ce convertisseur transforme les tensions analogiques fournies par le capteur en une valeur entière comprise entre 0 et 4095, puisque l'ESP32 utilise un ADC sur 12 bits.

Cependant, pour que ces données soient interprétables médicalement, il est nécessaire de les convertir en unités physiques réelles : le millivolt (mV), qui est l'unité de référence utilisée pour décrire les signaux ECG. En effet, les différentes ondes du tracé ECG vu dans le chapitre 1 présentent généralement les amplitudes suivantes :

- **Onde P** : de 0,05 à 0,25 mV
- **Complexe QRS** : de 1 à 2,5 mV
- **Onde T** : généralement inférieure à 1 mV

Sans cette conversion, les valeurs brutes issues de l'ADC n'ont pas de signification physiologique directe.

Le capteur AD8232 inclut un amplificateur intégré qui augmente considérablement l'amplitude du signal d'origine, afin de la rendre mesurable par l'ADC. Le gain typique de cet amplificateur est d'environ 1100**.** Par conséquent, il est indispensable de diviser la valeur obtenue par ce gain pour retrouver l'amplitude réelle du signal physiologique.

Formule de conversion appliquée :

La conversion finale en millivolts est donc réalisée en deux étapes :

1. Convertir la valeur ADC en tension (volts) :

$$\text{Vmesure} = \frac{\text{valeurADC} \times \text{Vref}}{4095} \qquad (2.1)$$

2. Diviser cette tension par le gain du capteur, puis convertir en millivolts :

$$\text{VECG(mV)} = \left(\frac{\text{Vmesure}}{\text{Gain}}\right) \times 1000 \quad (2.2)$$

Soit, en une seule formule :

$$\text{VECG(mV)} = \frac{\text{valeurADC} \times \text{Vref}}{4095 \times Gain} \times 1000 \quad (2.3)$$

En remplaçant par les valeurs typiques (Vref = 3.3 V, Gain ≈ 1100) :

$$VECG(mV) = \frac{valeurADC \times 3.3}{4095 \times 1100} \times 1000 \quad (2.4)$$

Ce traitement permet d'obtenir des données ECG immédiatement exploitables, sans nécessiter de recalculer côté client. Cela facilite l'analyse du signal, assure une cohérence entre acquisition**,** transmission et visualisation**,** et permet d'enregistrer en base de données des valeurs standardisées et prêtes à être interprétées par un professionnel de santé.

3.3.6 Traitement des données des autres capteurs environnementaux :

Dans notre système de santé connecté, seul le capteur ECG AD8232 nécessite une conversion analogique pour obtenir un signal interprétable médicalement. Les autres capteurs : DHT22, MAX30102 et MQ-135 fournissent directement des valeurs numériques dans leurs unités respectives (°C, %, BPM, %SpO_2, ppm). Grâce aux circuits intégrés ou à un calibrage logiciel, ces données sont envoyées telles quelles au serveur. Cette méthode permet une intégration plus simple, allège le traitement côté serveur, et garantit une compatibilité immédiate avec les interfaces utilisateur (web et mobile). Le traitement des données de ces capteurs environnementaux a été conçu pour privilégier la simplicité, la rapidité de transmission et la clarté des données affichées, tout en maintenant une cohérence avec les standards d'un système de santé connecté moderne.

3.3.7 Schéma de câblage :

Le schéma de câblage ci-dessous 3.9 illustre les connexions physiques entre la carte **ESP32** et l'ensemble des capteurs intégrés au projet. Il permet de visualiser clairement où sont reliés chaque capteur (AD8232, DHT22, MAX30102, MQ-135) ainsi que les connexions d'alimentation, de masse et de communication (broches analogiques ou numériques, bus I2C, etc.). Ce schéma constitue un guide essentiel pour reproduire le montage matériel sur une maquette réelle ou sur une plaque d'essai (breadboard).

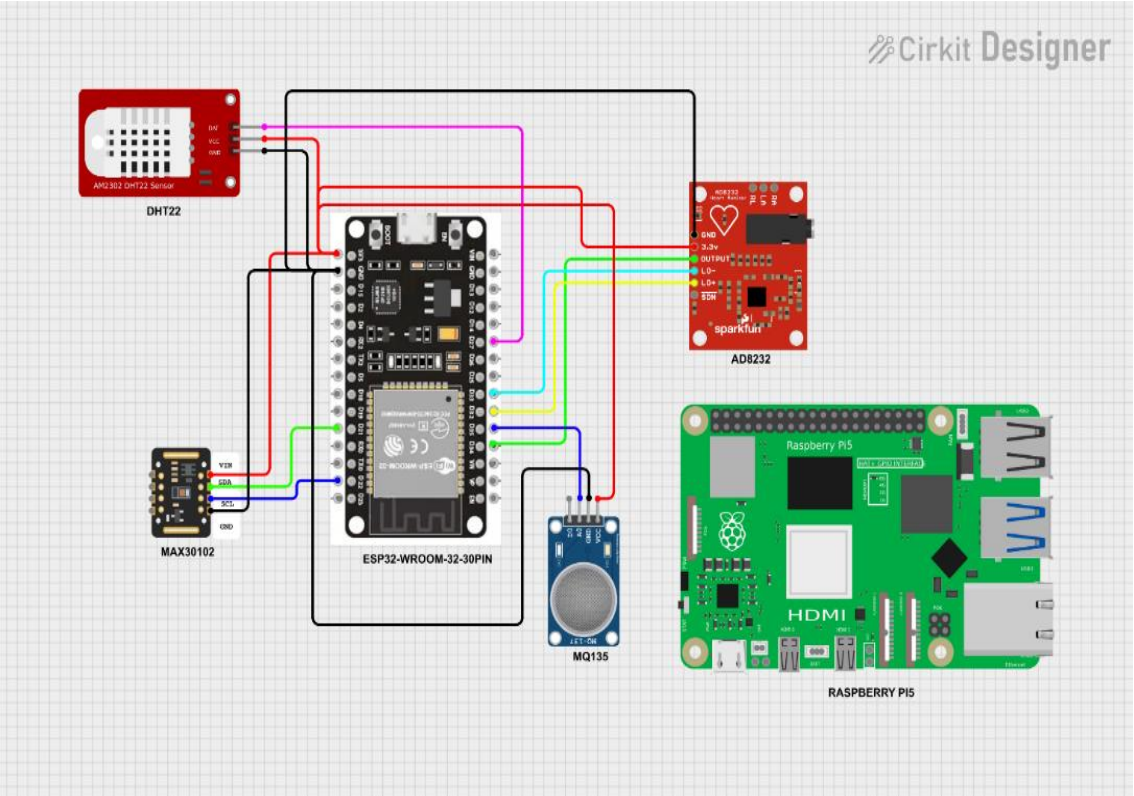


Figure 3.9 : Schéma de câblage complet du système de santé connecté

3.3.8 Schéma électrique:

En complément du schéma de câblage, un schéma électrique illustre par la figure 3.10 a été élaboré.

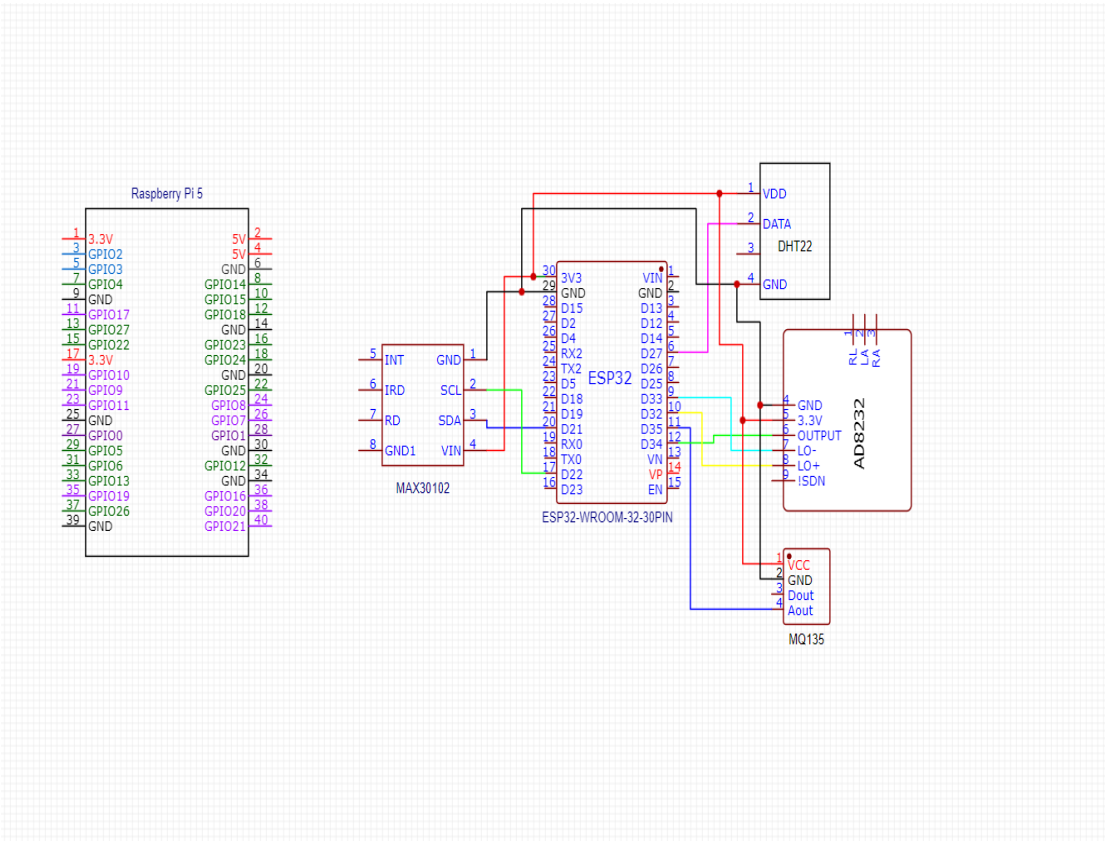


Figure 3.10 : Schéma électrique complet du système de santé connecté

3.3.9 Conception du circuit imprimé (PCB)

Afin d'assurer une intégration matérielle propre, compacte et durable des différents composants du système, un circuit imprimé PCB a été conçu à l'aide de la plateforme EasyEDA. Ce schéma regroupe l'ensemble des connexions entre l'ESP32 et les capteurs. La figure suivante 3.11 illustre la disposition finale du circuit sur une carte double face, optimisée pour un usage embarqué :

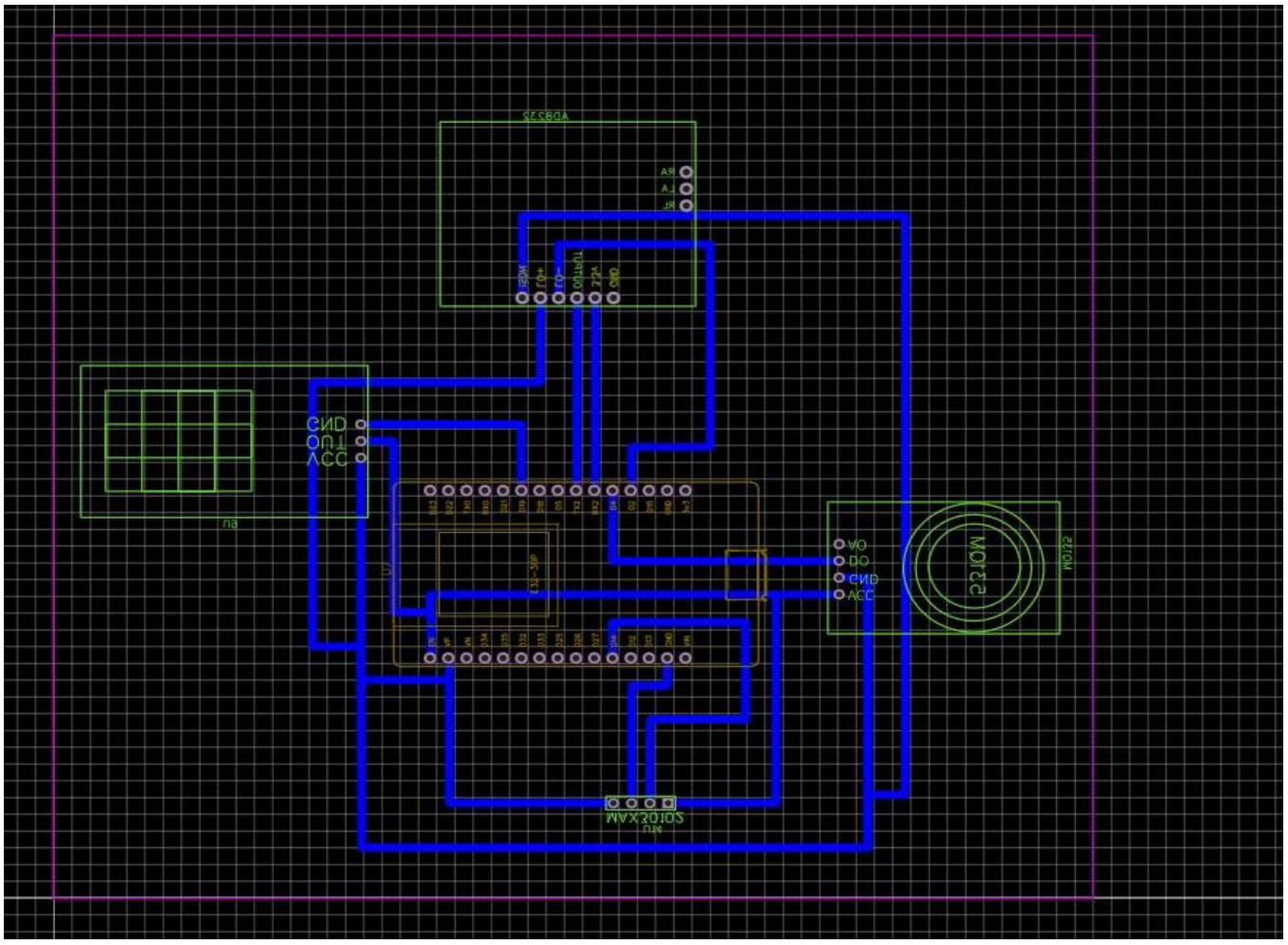


Figure 3.11 : Schéma du circuit imprimé (PCB) conçu sous EasyEDA

3.4 Partie logiciels :

La partie logicielle constitue l'élément central du système de santé connecté. Elle assure la communication entre les composants, le traitement et l'enregistrement des données, ainsi que leur affichage en temps réel via des interfaces conviviales. Le système repose sur une architecture distribuée intégrant plusieurs modules : un serveur Flask installé sur Raspberry Pi 5, une base de données SQLite, une interface web en HTML/CSS/JS, une application mobile Flutter. Un système de notification permet également de détecter les anomalies physiologiques. Cette section présente l'ensemble de ces composants logiciels et leur rôle dans le fonctionnement global du système.

3.4.1 Protocoles de communication

Utilisation du Wi-Fi pour la transmissions des données :

Pour assurer la communication entre le microcontrôleur ESP32 et le serveur Flask hébergé sur Raspberry Pi 5 nous avons opté pour **le Wi-Fi 802.11** comme technologie de connexion. Ce choix est justifié par plusieurs avantages :

- **Compatibilité native** : L'ESP32 intègre nativement une connectivité Wi-Fi, sans module additionnel.
- **Débit élevé** : Le Wi-Fi offre une bande passante suffisante pour transmettre en temps réel des flux de données tels que l'ECG.
- **Portée adéquate** : Le Wi-Fi couvre un environnement domestique ou hospitalier sans difficulté.
- **Intégration réseau** : Il permet au microcontrôleur d'envoyer les données directement au serveur local ou exposé via Ngrok (qu'on verra plus tard dans ce chapitre), sans passerelle intermédiaire.

Cette solution est donc parfaitement adaptée aux exigences du projet, notamment en termes de simplicité de mise en œuvre, stabilité de la connexion et rapidité de transmission.

Comparaison des technologies de communication sans fil utilisées en systèmes embarqués :
Avant de valider l'intégration du Wi-Fi comme solution de communication principale dans notre système, il est pertinent de comparer les différentes technologies sans fil couramment utilisées dans les systèmes embarqués. Le tableau ci-dessous **3.4** présente une synthèse des principales options (Wi-Fi, Bluetooth, LoRa, Zigbee, NFC, 4G/5G) selon des critères clés tels que le débit, la portée, la consommation énergétique ou encore les cas d'usage typiques.

Tableau 3.4 : Comparaison des technologies de communication sans fil utilisées en systèmes embarqués

Critère	Wi-Fi	Bluetooth	LoRa	Zigbee	NFC	4G/5G (cellulaire)
Débit de transmission	Élevé (jusqu'à 150 Mbps)	Faible à moyen (1–3 Mbps)	Très faible (quelques kbps)	Faible (20–250 kbps)	Très faible (<500 kbps)	Élevé (100 Mbps à >1 Gbps)
Portée	Moyenne (20–100 m)	Courte (1–10 m)	Très longue (jusqu'à 15 km)	Moyenne (10–100 m)	Très courte (<10 cm)	Très longue (jusqu'à 35 km)
Consommation énergétique	Moyenne à élevée	Très faible	Très faible	Faible	Très faible	Moyenne à élevée
Latence	Faible	Très faible	Élevée	Moyenne	Très faible	Moyenne
Installation / Infrastructure	Nécessite un réseau Wi-Fi	Très simple (pairing direct)	Nécessite une passerelle LoRa	Besoin d'un coordinateur réseau	Aucun réseau nécessaire	Carte SIM et abonnement requis
Utilisation typique	Réseaux domestiques, IoT local	Wearables, écouteurs, souris	IoT en zones rurales, agriculture	Domotique, capteurs industriels	Paiement sans contact, badges	IoT mobile, télésanté à distance

D'après le tableau 3.4, on remarque que le protocole Wi-Fi offre une solution simple, fiable et évolutive pour assurer la communication entre l'ESP32 et le serveur Flask. Cette approche, couramment utilisée dans les systèmes embarqués connectés, garantit une transmission efficace et une intégration fluide avec les interfaces web et mobile de notre projet.

3.4.2 Transmission des données via le protocole http

Le protocole retenu pour la communication entre l'ESP32 et le serveur central est **HTTP (HyperText Transfer Protocol)**. Il s'agit d'un protocole de communication client-serveur basé sur des requêtes POST, largement utilisé dans les architectures web modernes.

Ce choix se justifie par plusieurs raisons :

- **Simplicité du modèle client-serveur** : L'ESP32 agit comme client, envoyant périodiquement les données capteurs au serveur Flask via HTTP POST en format JSON.
- **Intégration directe avec Flask** : Le serveur Flask est structuré pour recevoir ces données via des routes spécifiques : `/ecg` pour les données ECG, et `/sensor_data` pour les autres paramètres vitaux.
- **Extensibilité** : Il est facile d'ajouter de nouveaux capteurs ou fonctionnalités sans modifier l'architecture réseau.
- **Lisibilité et débogage** : Les données échangées au format JSON sont facilement lisibles et analysables, facilitant les tests et la maintenance.

3.4.3 Système d'exploitation utilisé pour l'hébergement du serveur

Pour ce projet, nous avons choisi Raspberry Pi OS avec interface graphique comme le montre la figure 3.12 au lieu d'une version Lite en SSH ou un autre système d'exploitation comme Ubuntu. Ce choix s'explique par plusieurs raisons :

- **Compatibilité optimale** avec le matériel du Raspberry Pi.
- **Facilité de développement** local grâce à l'interface graphique (édition de fichiers, test du site web, gestion de la base de données SQLite).
- **Installation simplifiée** des outils nécessaires (Flask, SQLite, bibliothèques Python, etc.).

Cette version offre une meilleure ergonomie de travail, tout en étant parfaitement adaptée aux besoins techniques du système embarqué Cependant, il est important de noter que pour un déploiement réel et professionnel**,** une version Lite avec SSH activé est recommandée, car elle offre de meilleures performances**,** une sécurité renforcée et une meilleure gestion à distance du serveur

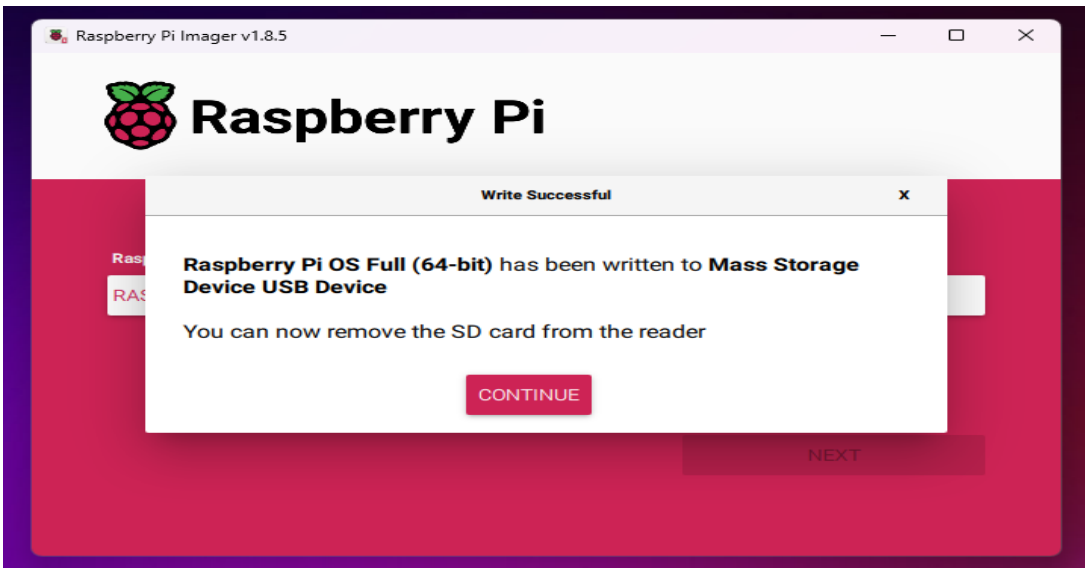


Figure 3.12 : Installation de Raspberry Pi OS via Raspberry Pi Imager

3.4.4 Serveur Flask

Dans notre architecture logicielle globale, le serveur joue un rôle central : il reçoit les données physiologiques en temps réel envoyées par l'ESP32, les enregistre dans une base de données SQLite, puis les met à disposition de la page web et de l'application mobile. Pour concevoir ce serveur, nous avons opté pour **Flask**, un microframework Python à la fois léger, flexible et particulièrement bien adapté à la création d'API RESTful.

a) Installation et développement de l'environnement du serveur

Après avoir ouvert un terminal, nous avons d'abord mis à jour les paquets du système à l'aide de la commande <sudo apt update>. Ensuite, nous avons installé pip3, le gestionnaire de paquets Python, avec <sudo apt install python3-pip>. Une fois pip3 installé, nous avons pu ajouter Flask ainsi que flask-cors (nécessaire pour autoriser les requêtes cross-origin, utilisées par les clients web et mobile) grâce à la commande pip3 <install flask flask-cors>.

b) Création de l'arborescence du projet

Après l'installation, nous avons structuré notre projet serveur de manière claire et modulaire. Le dossier principal, nommé **serveur_flask**, contient le fichier **app.py**, qui renferme l'ensemble du code du serveur Flask : définition des routes, traitement des données et gestion de la base de données. À ce même niveau se trouve également le fichier **ecg_data.db**, la base de données SQLite dans laquelle sont stockées les données (qu'on va aborder plus tard dans ce chapitre).

Deux sous-dossiers optionnels peuvent être ajoutés :

- **templates** : pour accueillir de potentiels fichiers HTML,
- **static** : pour les fichiers statiques (CSS, JavaScript, images).

Cette organisation simple garantit une bonne lisibilité et une maintenance facilitée du projet.

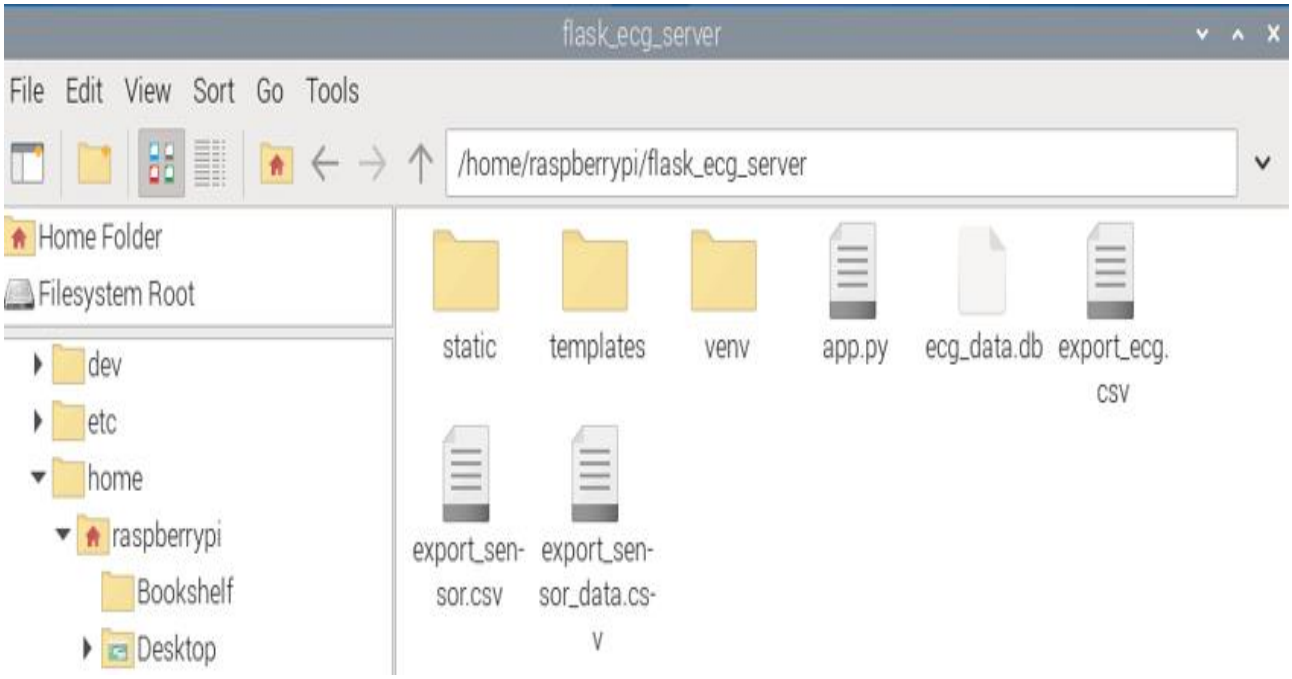


Figure 3.13 : Arborescence du projet Flask sur le Raspberry Pi 5

c) Création des routes API

Deux routes principales ont été mises en place dans le serveur afin de recevoir les données envoyées par l'ESP32.

- La première, **/ecg**, est dédiée à la réception des valeurs issues du capteur ECG. À chaque appel POST, le serveur extrait la valeur ECG depuis le corps de la requête (au format JSON), enregistre un horodatage en temps réel (via la fonction **datetime.now**()), puis insère ces données dans la table **ecg** de la base de données SQLite à l'aide d'une requête INSERT INTO. La connexion à la base est ouverte pour chaque insertion, puis immédiatement refermée afin de garantir l'intégrité des écritures.
- La seconde route, **/sensor_data**, est conçue pour recevoir les données issues des autres capteurs, notamment la température, l'humidité, la saturation en oxygène (SpO_2), la fréquence cardiaque (BPM) et la qualité de l'air. Ces cinq valeurs, extraites également d'un

corps JSON, sont stockées avec un horodatage dans la table **sensor_data** de la base SQLite. Le traitement suit la même logique : ouverture de la connexion, exécution de la requête, puis fermeture propre de la connexion.

```python
@app.route('/ecg', methods=['POST'])
def recevoir_ecg():
    data = request.get_json()
    if data and 'ecg_value' in data:
        ecg_valeur = data['ecg_value']
        timestamp = datetime.now().isoformat()

        # Insertion dans SQLite
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute("INSERT INTO ecg (timestamp, e
        conn.commit()
        conn.close()
```

Figure 3.14 : Code serveur de la route /ecg

Ces routes sont appelées toutes les **50 millisecondes** pour les données ECG, et toutes les **3 secondes** pour les autres capteurs. Elles permettent un enregistrement fiable et structuré des mesures physiologiques, tout en assurant une communication fluide entre l'ESP32 et le serveur (pour ne pas saturer le serveur).

d) Mise en marche du serveur Flask sur Raspberry Pi

Une fois la logique serveur entièrement codée dans le fichier **app.py**, la mise en route se fait très simplement. Depuis un terminal, en se plaçant dans le répertoire **/serveur_flask/**, il suffit ensuite d'exécuter la commande <`python3 app.py`>. Cela lance le serveur Flask en local, accessible à partir de l'adresse IP du Raspberry Pi 5 (par exemple http://192.168.112.105:5000). Lors de son exécution, le terminal affiche des messages de log qui confirment que le serveur est bien en écoute, prêt à recevoir des requêtes HTTP de la part de l'ESP32. Ces logs sont également très utiles dans notre phase de développement, car ils permettent un **débogage en temps réel** : chaque requête entrante, chaque valeur reçue, ou chaque erreur potentielle est immédiatement affichée dans la console.

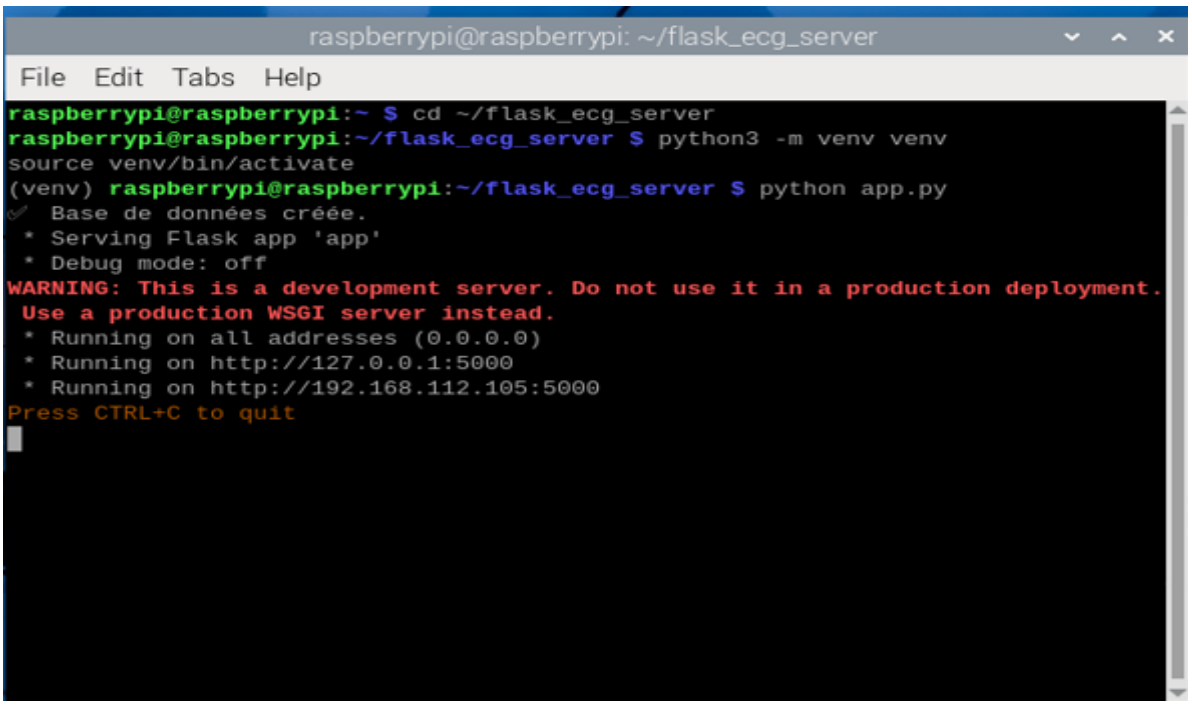


Figure 3.15 : Mise en marche du serveur

3.4.5 Mise en place de la base de données SQLite

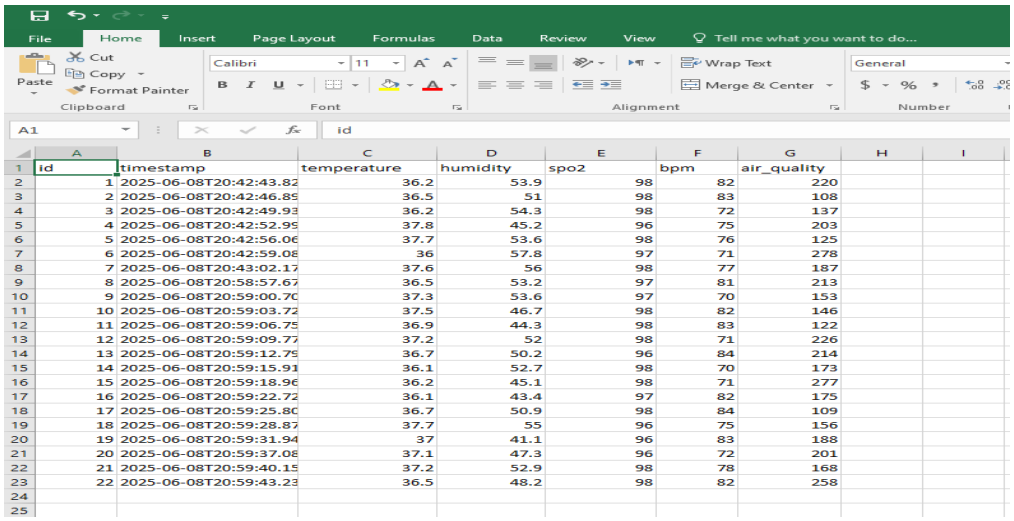
Afin de stocker durablement les données reçues depuis l'ESP32, nous avons opté pour l'utilisation de **SQLite**, une base de données légère, simple à configurer et parfaitement adaptée aux environnements embarqués comme le Raspberry Pi 5. SQLite ne nécessite aucun serveur de gestion séparé : les données sont enregistrées directement dans un fichier local (dans notre cas, ecg_data.db), ce qui rend son intégration à un projet Flask à la fois rapide et efficace.

Dès la création du projet, nous avons généré ce fichier SQLite à la racine de notre dossier serveur_flask. Ce fichier contient deux tables principales : ecg et sensor_data. La table ecg est destinée à stocker les valeurs issues du capteur ECG, avec deux colonnes essentielles : une pour la valeur mesurée, et une autre pour l'horodatage correspondant. Quant à la table sensor_data, elle enregistre les données envoyées par les autres capteurs, avec les colonnes suivantes : temperature, humidity, spo2, bpm, air_quality, et timestamp.

Ces deux tables ont été créées manuellement à l'aide de requêtes SQL exécutées directement dans un petit script Python exécuté une seule fois pour initialiser la base. Une fois ce script lancé, le fichier ecg_data.db a été généré automatiquement, prêt à recevoir les données via les requêtes POST envoyées aux routes /ecg et /sensor_data.

Chaque fois qu'une requête est reçue, le serveur Flask se connecte temporairement à la base de

données, insère les nouvelles données en utilisant une requête ‘’INSERT INTO’’, puis referme la connexion. Cette méthode garantit la stabilité du système, tout en conservant un historique complet des données mesurées, consultables ultérieurement par la page web ou l’application mobile.



id	timestamp	temperature	humidity	spo2	bpm	air_quality
1	2025-06-08T20:42:43.82	36.2	53.9	98	82	220
2	2025-06-08T20:42:46.85	36.5	51	98	83	108
3	2025-06-08T20:42:49.93	36.2	54.3	98	72	137
4	2025-06-08T20:42:52.95	37.8	45.2	96	75	203
5	2025-06-08T20:42:56.06	37.7	53.6	98	76	125
6	2025-06-08T20:42:59.08	36	57.8	97	71	278
7	2025-06-08T20:43:02.17	37.6	56	98	77	187
8	2025-06-08T20:58:57.67	36.5	53.2	97	81	213
9	2025-06-08T20:59:00.70	37.3	53.6	97	70	153
10	2025-06-08T20:59:03.72	37.5	46.7	98	82	146
11	2025-06-08T20:59:06.75	36.9	44.3	98	83	122
12	2025-06-08T20:59:09.77	37.2	52	98	71	226
13	2025-06-08T20:59:12.79	36.7	50.2	96	84	214
14	2025-06-08T20:59:15.91	36.1	52.7	98	70	173
15	2025-06-08T20:59:18.96	36.2	45.1	98	71	277
16	2025-06-08T20:59:22.72	36.1	43.4	97	82	175
17	2025-06-08T20:59:25.80	36.7	50.9	98	84	109
18	2025-06-08T20:59:28.87	37.7	55	96	75	156
19	2025-06-08T20:59:31.94	37	41.1	96	83	188
20	2025-06-08T20:59:37.08	37.1	47.3	96	72	201
21	2025-06-08T20:59:40.15	37.2	52.9	98	78	168
22	2025-06-08T20:59:43.23	36.5	48.2	98	82	258

Figure 3.16 : Contenu du fichier csv de la base de donnée des capteurs.

Manipulation de la base de données SQLite :

La manipulation de la base de données SQLite a permis de vérifier l’enregistrement correct des données envoyées par l’ESP32. Grâce à l’outil en ligne de commande sqlite3, nous avons pu accéder à la base ecg_data.db, explorer sa structure avec les commandes .schema ecg et .schema sensor_data, et consulter les dernières entrées via des requêtes SQL telles que SELECT * FROM ecg ORDER BY timestamp DESC LIMIT 10. Ces opérations ont été essentielles pour valider le bon fonctionnement du système lors des phases de test et pour corriger d’éventuelles incohérences avant l’intégration des interfaces.

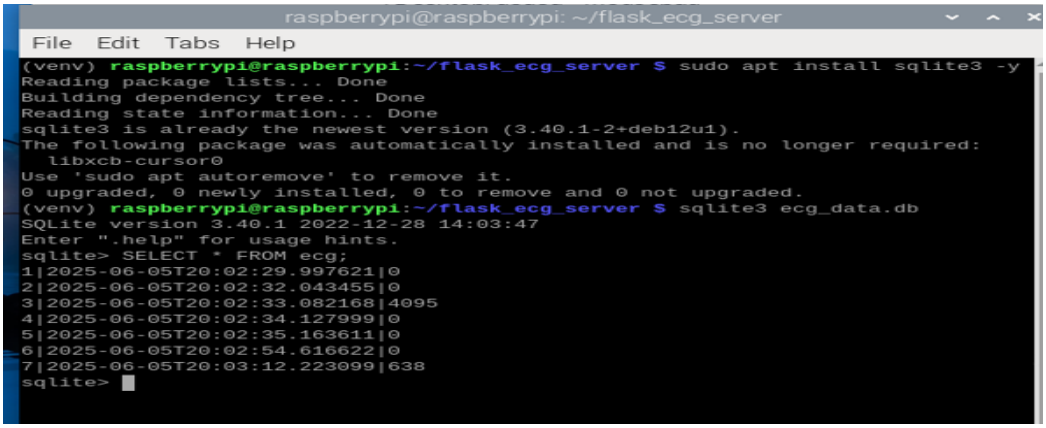


Figure 3.17 : Consultation de la base de donnes via le terminal

3.4.6 Développement de la page web (interface cliente)

L'une des composantes clés de notre système de santé connecté est **l'**interface web, qui permet de visualiser en temps réel les données issues des différents capteurs. Cette page web joue un rôle essentiel pour les soignants ou les utilisateurs souhaitant accéder rapidement à l'évolution des paramètres vitaux.

a. Choix des technologies

Pour la réalisation de cette interface, nous avons opté pour une solution légère, simple et efficace : une **page HTML** intégrant du **JavaScript** pour les appels réseau, et **Chart.js** pour la représentation graphique des données. Ce choix nous permet de créer une interface moderne, interactive, et accessible depuis n'importe quel navigateur, sans nécessiter d'installation.

b. Structure de la page

La page web a été construite autour de **quatre sections principales**, offrant une interface claire et intuitive :

- **Dashboard** : Vue d'ensemble synthétique des paramètres vitaux, présentant en un coup d'œil les dernières valeurs mesurées de chaque capteur, ainsi que l'état global du patient.
- **ECG** : Affiche un **graphe en temps réel** du signal électrocardiographique, mis à jour en continu pour assurer une visualisation fluide et réactive.
- **Capteurs** : Liste les **dernières mesures** des capteurs de température, d'humidité, de saturation en oxygène (SpO_2), de fréquence cardiaque (BPM) et de qualité de l'air. Cette section propose également une **fonction d'exportation au format CSV**, permettant d'archiver ou d'analyser les données hors ligne.
- **À propos** : Présente le **logo de l'USTHB**, ainsi que le nom de la Faculté de Génie Électrique – Master 2 Électronique, spécialité Systèmes Embarqués**.**

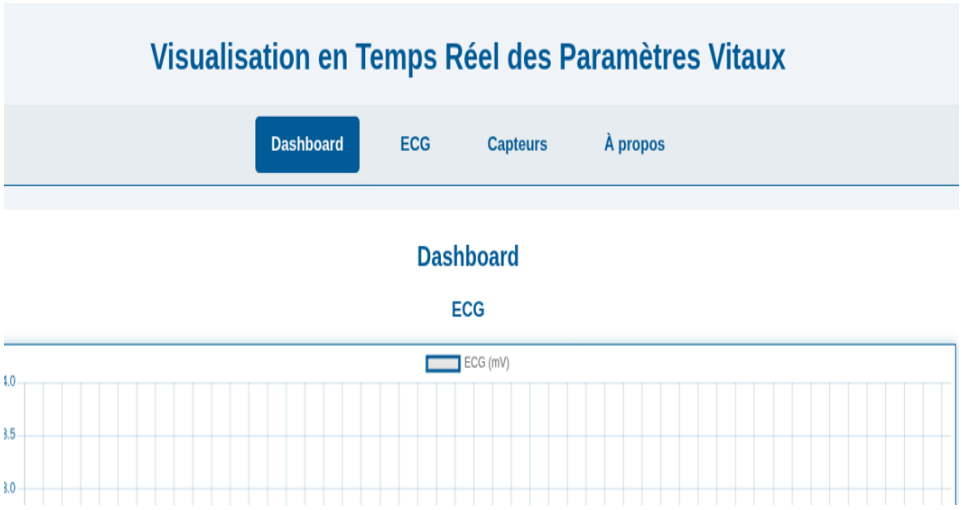


Figure 3.18 : Vue d'ensemble de la page web

La communication entre cette page et la base de données SQLite se fait indirectement via le **serveur Flask**, qui agit comme intermédiaire. Lorsqu'un utilisateur ouvre la page, le JavaScript intégré effectue des **requêtes HTTP GET** vers les routes définies dans l'API Flask (/get_ecg et /get_sensor_data). Ces routes se connectent à la base de données locale (fichier ecg_data.db sur le Raspberry Pi), extraient les données les plus récentes grâce à des requêtes SQL, puis renvoient les résultats au format JSON.

Le JavaScript côté client traite ensuite ces réponses JSON pour :

- Mettre à jour dynamiquement le graphe ECG toutes les 0,5 secondes avec les dernières valeurs.
- Afficher les données numériques des capteurs dans un tableau ou des cartes (cards) avec leur timestamp associé.
- Proposer un bouton d'export CSV, qui transforme les données reçues en fichier téléchargeable.
- Intégrer un bouton de réinitialisation du graphe ECG, permettant de vider l'affichage et de repartir sur une base propre, notamment utile lors d'un changement de patient ou d'un redémarrage de session.

c. Affichage en temps réel

Pour obtenir un affichage fluide, nous avons utilisé la fonction setInterval() de JavaScript afin d'interroger régulièrement les routes de l'API (toutes les 500 ms pour l'ECG, et toutes les 3

secondes pour les autres capteurs). Cela permet de simuler une visualisation continue, sans bloquer le navigateur ni surcharger le serveur.



Figure 3.19 : Graphe de L'ECG de la page WEB

Chaque nouvelle valeur reçue est insérée dans le graphique Chart.js, qui supprime automatiquement les plus anciennes pour conserver une fenêtre temporelle fixe (par exemple les 10 dernières secondes d'enregistrement). Cette méthode offre une **expérience visuelle claire et immédiate**, très adaptée pour le suivi en direct des constantes vitales.

Remarque : *Le signal ECG affiché dans cette version web est représentatif mais présente certaines limitations techniques. Afin de ne pas saturer le serveur Flask, un délai d'envoi a été volontairement introduit, ce qui peut entraîner une légère perte de précision dans la restitution du tracé. De plus, aucun filtrage du signal brut n'a été appliqué à ce stade. Pour une utilisation en conditions réelles, il serait nécessaire d'intégrer un filtrage numérique (notamment passe-bas/passe-haut) et de réduire la latence d'acquisition pour garantir une représentation fidèle du signal cardiaque.*

d. Interconnexion entre les composants

Le **lien entre la page web, le serveur Flask et la base SQLite** peut se résumer ainsi :

1. La page HTML est hébergée sur le Raspberry Pi dans le dossier static/ de Flask, ou consultée en local depuis un navigateur.
2. Le JavaScript embarqué dans cette page effectue des appels GET à l'API Flask.
3. Flask intercepte la requête, se connecte à la base de données SQLite (ecg_data.db), extrait les données nécessaires, puis les retourne au navigateur.

4. Le navigateur les affiche en temps réel via Chart.js ou dans des éléments HTML classiques.

Grâce à cette architecture, la page web fonctionne comme un véritable client léger connecté à l'API, capable de récupérer et afficher les données issues du microcontrôleur ESP32, **s**ans jamais accéder directement à la base SQLite**.** Cette séparation garantit une meilleure sécurité, une modularité claire, et une facilité d'évolution du projet.

3.4.7 Développement de l'application mobile Flutter

Dans une logique de système de santé connecté, il était primordial de proposer une interface mobile moderne et réactive, permettant aux utilisateurs (qu'il s'agisse de patients ou de personnel médical) de suivre en temps réel les paramètres vitaux. Pour cela, nous avons développé une application Flutter compatible Android, structurée autour d'un tableau de bord complet. L'application communique avec notre API Flask via des requêtes HTTP GET ciblant les routes /get_ecg et /get_sensor_data, respectivement pour les données électrocardiographiques et pour les valeurs issues des autres capteurs. Les réponses JSON renvoyées par le serveur permettent un traitement rapide et fluide des données côté Flutter. L'architecture générale de l'application repose sur plusieurs écrans, dont un Dashboard principal affichant les paramètres vitaux sous forme de graphes dynamiques. Le signal ECG est mis à jour toutes les 500 ms à l'aide du package fl_chart, tandis que les autres capteurs (température, humidité, SpO_2, BPM, qualité de l'air) sont actualisés toutes les 3 secondes et présentés sous forme de cartes informatives et de graphiques simples, permettant une lecture claire et intuitive de l'état de santé du patient.

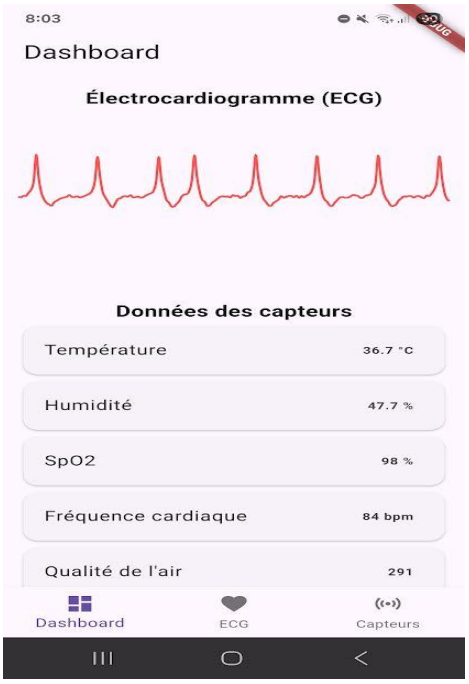


Figure 3.20 : Aperçu de l'application mobile Flutter en phase de développement

Notifications et alertes en temps réel dans l'application mobile Flutter

Après le développement et l'amélioration de l'application, une rubrique Alertes a été ajoutée pour signaler automatiquement les anomalies détectées (par exemple une saturation trop basse ou un rythme cardiaque élevé). Les alertes s'affichent sous forme de cartes colorées avec la nature de l'anomalie, la valeur mesurée et le moment de détection.

Enfin, une rubrique À propos présente le logo de l'USTHB ainsi que l'intitulé du diplôme : Faculté de Génie Électrique – Master 2 Électronique, spécialité Systèmes Embarqués, offrant un contexte académique clair au projet.



Figure 3.21 : Fenêtre pour modifier les seuils des capteurs

3.5 Passage du serveur local en réseau étendu (WAN) avec Ngrok

Initialement, le serveur Flask était hébergé localement sur le Raspberry Pi 5 et accessible uniquement en réseau local (LAN). Si cette configuration est adaptée pour un usage restreint et sécurisé, elle limite l'accès à distance. Pour y remédier, nous avons utilisé Ngrok, un outil de tunneling permettant d'exposer temporairement le serveur sur Internet (WAN) sans modifier la configuration réseau ni ouvrir de ports.

3.5.1 Transformation d'un serveur LAN en serveur WAN :

Dans un réseau local, les appareils utilisent des adresses IP privées (de type 192.168.x.x) pour communiquer entre eux, mais ces adresses ne sont pas accessibles depuis l'extérieur d'Internet sans configuration complexe, comme la redirection de ports sur le routeur. Pour contourner cette limitation, nous avons utilisé Ngrok, un outil qui permet de créer un tunnel sécurisé entre notre serveur local (hébergé sur le Raspberry Pi 5) et une URL publique accessible depuis n'importe où. L'installation de Ngrok s'est faite via quelques commandes simples adaptées à l'architecture ARM du Raspberry Pi :

1. Téléchargement de l'archive Ngrok : wget https://bin.equinox.io/c/4VmDzA7iaHb/ngrok-stable-linux-arm.zip
2. Décompression en utilisant la commande unzip ngrok-stable-linux-arm.zip
3. Instalation

Une fois Ngrok installé, nous avons lié notre Raspberry Pi à notre compte Ngrok avec la commande ngrok config add-authtoken VOTRE_TOKEN_NGROK
Ce token personnel garantit un tunnel stable et sécurisé. Enfin, pour exposer notre serveur Flask tournant sur le port 5000, nous avons lancé la commande : ngrok http 5000
Ngrok a généré une URL publique temporaire (par exemple https://abcd1234.ngrok.io) redirigeant automatiquement vers http://localhost:5000. Cette solution nous a permis d'accéder à notre API Flask depuis un navigateur, une application Flutter ou tout autre appareil connecté à Internet, sans modification du routeur ni ouverture de ports.

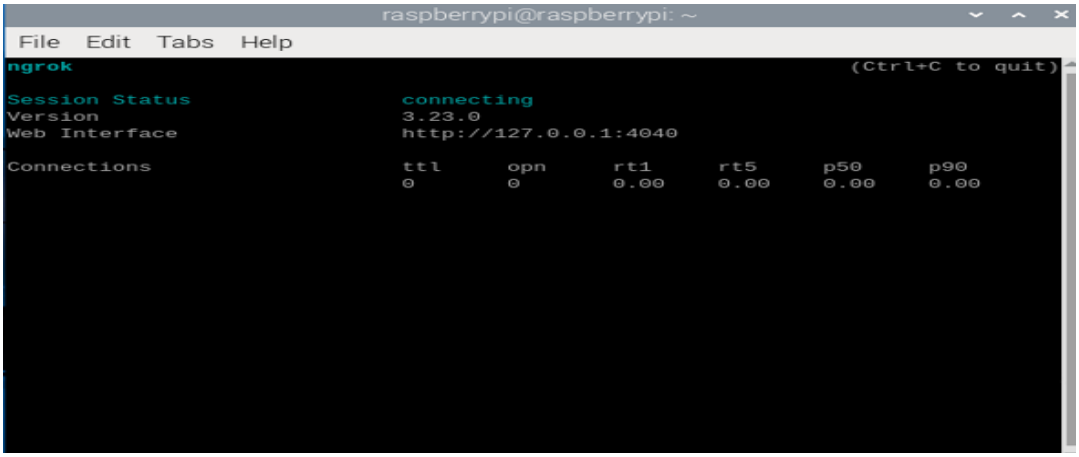


Figure 3.22 : Mise en marche de Ngrok à partir du terminal

Grâce à cette solution, nous avons pu **ouvrir temporairement notre système embarqué vers Internet**, facilitant ainsi l'accès à distance, les démonstrations publiques, ou encore les tests en mobilité, tout en évitant les contraintes de configuration réseau souvent imposées par les box Internet ou les pare-feux.

3.5.2Comparaison LAN vs WAN :

Le tableau suivant 3.5 met en évidence les différences clés entre l'utilisation d'un serveur local (LAN) **et** l'ouverture de ce serveur au réseau Internet via Ngrok (WAN). Il permet de comparer les deux approches selon plusieurs critères techniques et pratiques, afin d'illustrer les avantages et limitations de chacune dans le cadre de notre projet.

Tableau 3.5: Comparaison des modes d'accès au serveur : LAN local vs WAN

Critère	LAN (Local Area Network)	WAN avec Ngrok (Wide Area Network)
Accessibilité	Limitée au réseau local (Wi-Fi domestique)	Accessible depuis Internet (4G, autre Wi-Fi, etc.)
Configuration	Simple, aucune modification requise	Simple via Ngrok, pas besoin d'ouvrir les ports
Latence	Très faible (réseau interne)	Légèrement plus élevée (tunnel Internet)
Utilisation pour tests	Idéal en phase de développement local	Idéal pour démos, accès mobile, travail à distance
Stabilité	Très stable	Moins stable (sauf compte pro)
Sécurité	Moins exposé, donc plus sûr par défaut	Exposé à Internet, nécessite une sécurisation active

3.5.3 Sécurité et exposition des données

Exposer une API médicale sur Internet, même de façon temporaire via Ngrok, soulève naturellement des questions de **cybersécurité**. Même si l'URL est aléatoire et que la session est temporaire, la **surface d'attaque** est élargie. Pour réduire les risques, plusieurs précautions ont été prises :

- **Accès limité dans le temps** : les tunnels Ngrok gratuits ne restent actifs que durant la session, ce qui réduit les risques de compromission persistante.
- **Connexion HTTPS obligatoire** : toutes les requêtes passent par un tunnel chiffré TLS/SSL, garantissant la confidentialité des données en transit**.**
- **Protection par mot de passe** : pour sécuriser davantage l'accès, il est possible d'utiliser l'option -auth au moment du lancement du tunnel. Par exemple :ngrok http -auth="medecin:secret123" 5000

Ainsi, seuls les utilisateurs disposant du bon identifiant et mot de passe peuvent accéder à l'API.

Il est également envisageable de modifier le code Flask pour restreindre les requêtes entrantes à certaines adresses IP ou de mettre en place une authentification par token.

À terme, pour un usage en production, la solution la plus robuste serait de déployer le serveur Flask sur une infrastructure cloud sécurisée (type VPS, AWS, Azure) avec un nom de domaine dédié**,** certificat SSL propre, et une authentification forte (par exemple OAuth2).[25]

3.6 Test et Validation du fonctionnement global du système

Afin de confirmer la robustesse et la cohérence de l'architecture développée, nous avons procédé à une série de **tests pratiques complets** impliquant l'ensemble des composants du système, aussi bien du côté matériel (ESP32 et capteurs réels) que du côté logiciel (serveur Flask, base SQLite, page web en HTML/JS, application mobile Flutter).

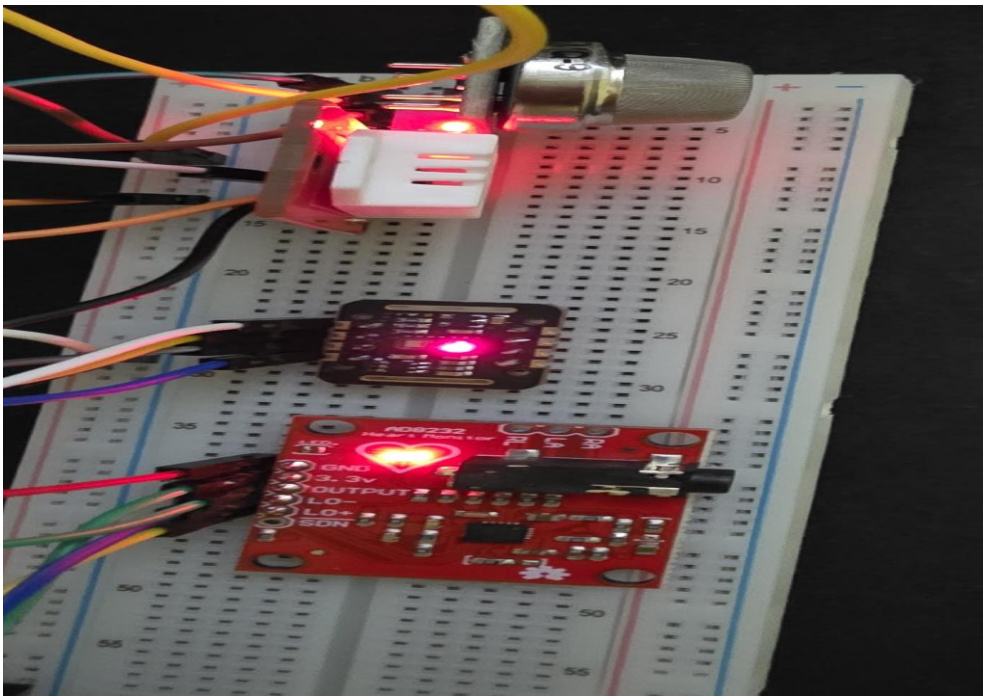


Figure 3.23 Tests du montage réel dans un lab d'essai

Nous avons d'abord validé le **bon fonctionnement de la phase d'acquisition** côté ESP32. Tous les capteurs ont été connectés correctement au microcontrôleur :

- ECG (AD8232)
- Température et humidité (DHT22)
- SpO2 + fréquence cardiaque (MAX30102)
- Qualité de l'air (MQ135)

Sur **l'**Arduino IDE**,** nous avons observé les valeurs lues en temps réel, confirmant que le Wi-Fi était bien fonctionnel et que les données étaient structurées au format JSON selon les routes définies (/ecg et /sensor_data).

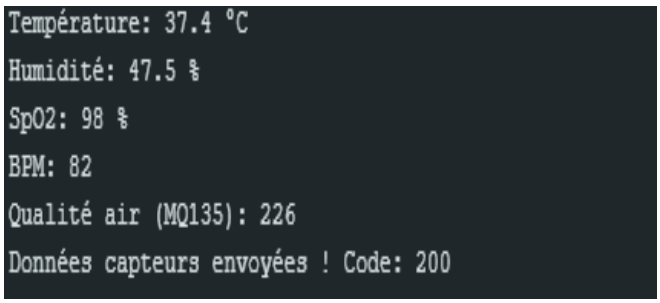


Figure 3.24 : données envoyée à partir de l'esp 32 sur serial monitor

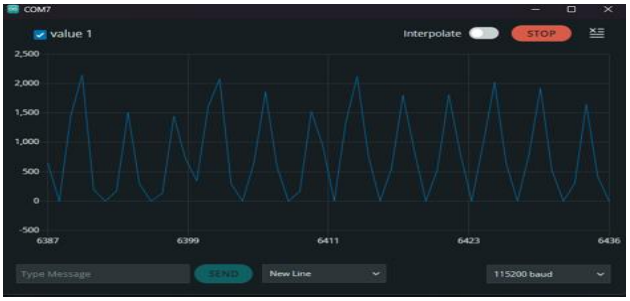


Figure 3.25 : Aperçu du signal ECG non filtrée sur serial plotter

Remarque :

Le signal ECG obtenu via le Serial Plotter de l'Arduino IDE correspond aux données brutes issues du capteur AD8232, sans aucun filtrage logiciel. Bien que cela permette une première visualisation du tracé cardiaque, ce signal peut être fortement affecté par des parasites, des mouvements ou du bruit électrique. Dans le cadre d'une utilisation médicale ou d'un traitement plus rigoureux, il est donc indispensable d'intégrer des filtres numériques (par exemple passe-bas, passe-haut ou filtre de Notch) pour extraire les composantes pertinentes du signal (ondes P, QRS, T) et améliorer la fiabilité de l'analyse.

Tests côté serveur Flask et base SQLite (Raspberry Pi 5)

Nous avons ensuite vérifié la réception correcte des données côté serveur Flask**,** hébergé sur le Raspberry Pi 5. Les routes ont été testées manuellement via Postman, puis en conditions réelles avec l'ESP32. L'interface en ligne de commande du serveur a confirmé la réception continue des paquets JSON, avec un enregistrement immédiat dans la base de données SQLite.

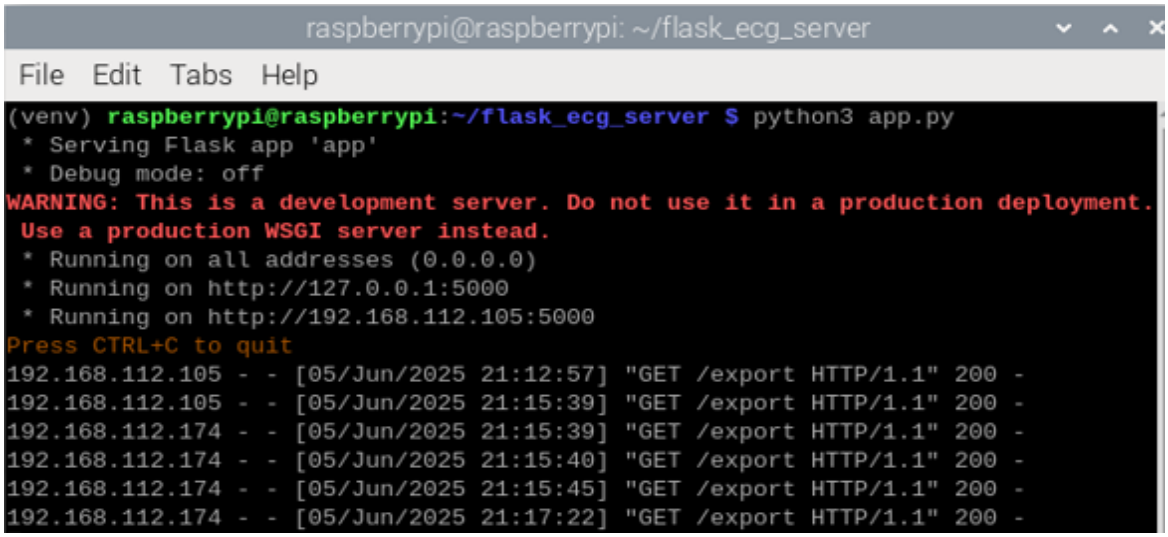


Figure 3.26 : Résultat de l'envoi des requêtes http sur le serveur

Tests d'affichage sur la page web HTML/JS

La page web a été ouverte dans un navigateur local. Nous avons observé :

- La courbe ECG s'afficher en temps réel et évoluer dynamiquement.
- Les valeurs numériques des autres capteurs (température, humidité, SpO2, BPM, qualité de l'air) mises à jour toutes les 3 secondes.
- Un rafraîchissement fluide et sans coupure, grâce à l'utilisation de setInterval() et Chart.js.

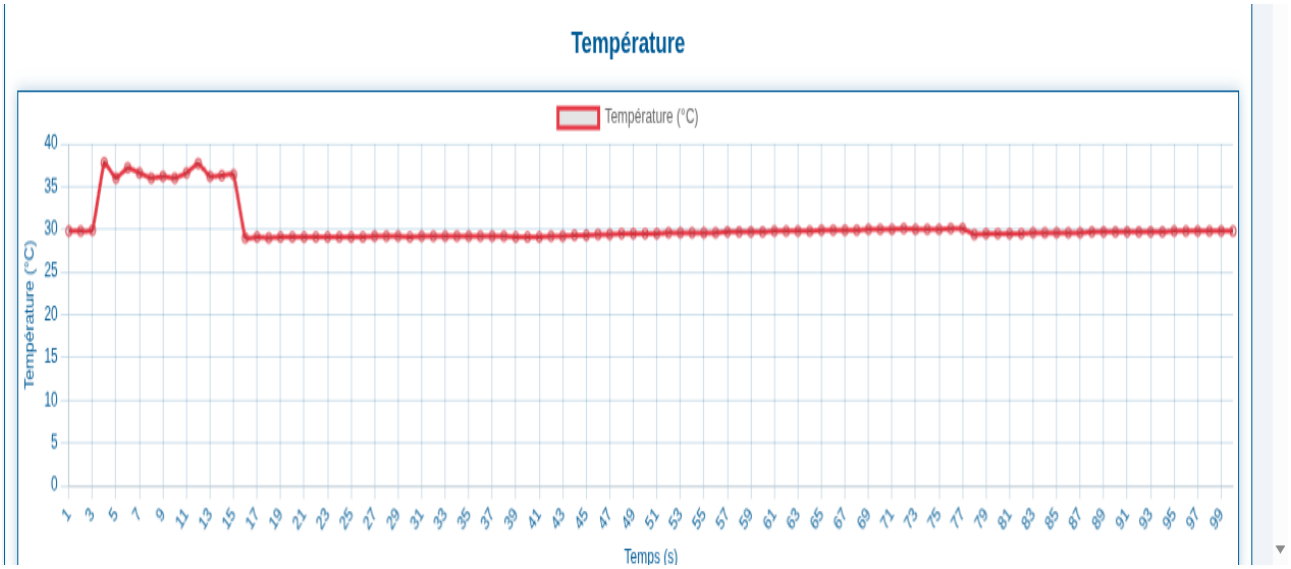


Figure 3.27 : Exemple d'affichage de la courbe température sur la page WEB

Tests de l'application mobile Flutter

Enfin, nous avons lancé l'application mobile sur un smartphone Android. L'application a correctement :

- Affiché le **graphe ECG en temps réel**.
- Mis à jour les **autres données de capteurs** toutes les 3 secondes.

- Affiché l'état de connexion à l'API (locale ou via Ngrok) sans latence excessive.



Figure 3.28 : Page à propos de l'application Flutter

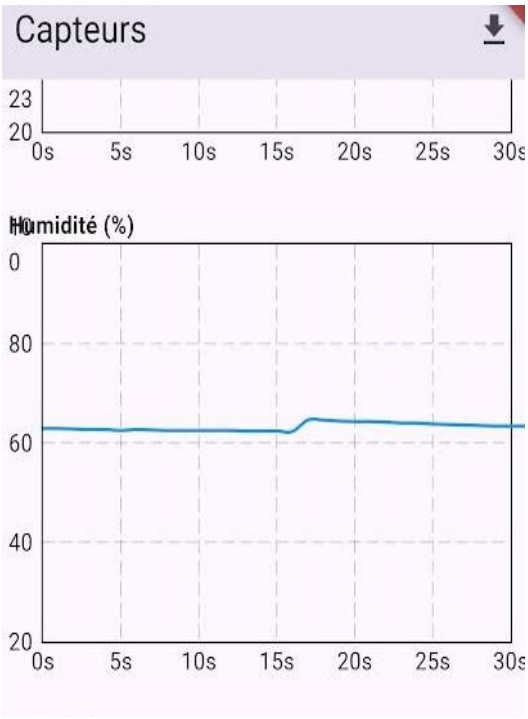


Figure 3.29 : Exemple d'affichage de la courbe humidité dans l'application flutter

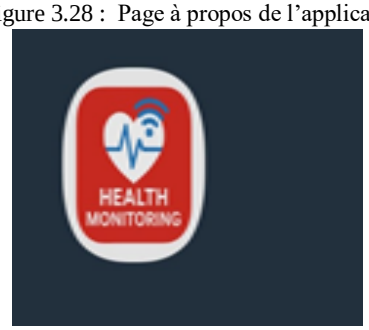


Figure 3.30 : Icone de l'application mobile flutter

Tests d'accessibilité via Ngrok

Pour valider la **portée WAN** du système, nous avons utilisé Ngrok afin de créer une URL publique vers notre serveur Flask local. Nous avons vérifié l'accès depuis :

- Un smartphone en 4G
- Un autre PC connecté à un réseau Wi-Fi différent

Les tests ont confirmé que la page web et l'application Flutter pouvaient interroger l'API via l'adresse `https://xxxxxx.ngrok.io` sans modification de code côté client.

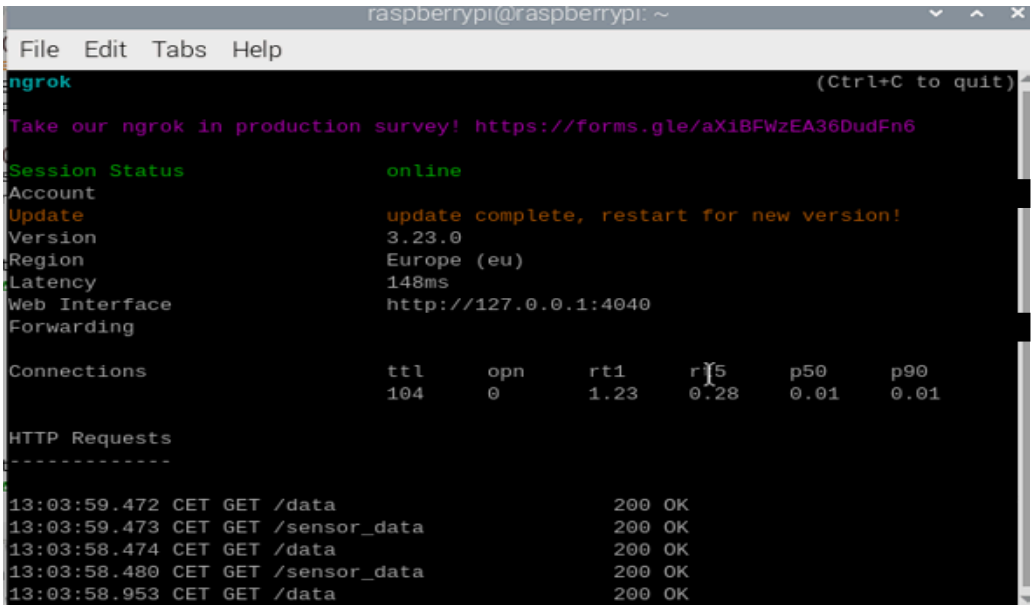


Figure 3.31 : Connexion du serveur local a internet via Ngrok

3.7 Résultats et conclusion :

Les différents tests effectués valident le bon fonctionnement du système dans son ensemble. Ils ont permis de vérifier l'acquisition effective des données depuis les capteurs, leur transmission fluide via le protocole HTTP, leur enregistrement automatique dans une base de données SQLite, ainsi que leur affichage en temps réel sur l'interface web et l'application mobile. Le système a également démontré sa capacité à être accessible localement (via réseau Wi-Fi) et à distance (grâce à l'utilisation de Ngrok), ce qui confirme sa pertinence pour des cas réels de télésurveillance médicale.

L'architecture développée repose sur une conception modulaire, robuste et évolutive. L'ESP32 agit comme une passerelle intelligente, collectant les données de plusieurs capteurs embarqués : le capteur ECG AD8232 (avec un échantillonnage toutes les 50 ms), le DHT22 (température et humidité), le MAX30102 (SpO_2 et rythme cardiaque), et le MQ-135 (qualité de l'air), ces derniers transmettant leurs données toutes les 3 secondes. Les données sont envoyées au serveur Flask, installé sur un Raspberry Pi 5, via des requêtes HTTP POST sur deux routes distinctes (/ecg et /sensor_data), puis stockées dans une base SQLite.

Côté visualisation, la page web HTML/JavaScript interroge automatiquement le serveur à intervalles réguliers (500 ms pour l'ECG et 3 s pour les autres capteurs) afin d'afficher des graphiques mis à jour en continu grâce à Chart.js. L'application mobile Flutter adopte la même logique, en présentant les données sous forme de tableaux de bord dynamiques et interactifs.

Enfin, pour garantir l'accessibilité hors réseau local, un tunnel sécurisé a été mis en place avec Ngrok, permettant d'exposer le serveur Flask à une URL publique, sans avoir à modifier la configuration du routeur. Cette solution simplifie grandement l'accès distant, tout en assurant la sécurité de la communication.

En résumé, cette architecture intégrée offre un système fiable, adaptable, et prêt à l'emploi pour des applications concrètes de santé connectée. Elle pose également les bases pour des extensions futures, comme l'ajout de notifications intelligentes, d'alertes en cas d'anomalies, ou encore d'algorithmes d'analyse avancée des signaux.

Conclusion générale

Dans un contexte où les technologies de l'Internet des Objets Médicaux (IoMT) prennent une place croissante dans l'amélioration des soins de santé, ce mémoire a présenté la conception, la mise en œuvre et la validation d'un système embarqué capable de surveiller en temps réel plusieurs paramètres physiologiques. À travers l'intégration de capteurs biomédicaux (ECG, température, humidité, SpO2, fréquence cardiaque et qualité de l'air), d'un microcontrôleur ESP32, d'un serveur local basé sur Flask, ainsi que d'interfaces utilisateur web et mobile, ce travail a permis de construire une solution cohérente, légère et fonctionnelle.

La démarche suivie s'est articulée autour de plusieurs axes : d'abord la compréhension des signaux vitaux, puis le choix des composants matériels adaptés, suivi du développement logiciel embarqué, de la mise en place d'une base de données locale et d'un serveur HTTP, et enfin la conception d'interfaces dynamiques facilitant la visualisation des données en temps réel.

Le système proposé a démontré sa capacité à collecter, transmettre, stocker et afficher des données biométriques avec une latence faible et une architecture modulaire, permettant une éventuelle extension ou intégration de traitements avancés (tels que l'intelligence artificielle ou l'analyse prédictive) dans le futur.

Les résultats obtenus sont encourageants, notamment en matière de fiabilité des communications, de stabilité du serveur local et de fluidité d'affichage sur les différentes interfaces. Toutefois, certaines perspectives d'amélioration demeurent, notamment en ce qui concerne la miniaturisation, l'autonomie énergétique, le filtrage du signal ECg ou encore l'intégration de mécanismes de sécurité renforcés pour les données de santé.

Ainsi, ce travail s'inscrit dans une logique de modernisation des outils de télésurveillance médicale, en montrant qu'il est possible, avec des moyens accessibles et une approche méthodique, de proposer des solutions efficaces et adaptables aux besoins concrets du terrain médical.

BIBLIOGRAPHIE:

[1] “Système cardiovasculaire (circulatoire),” Kenhub. Available: https://www.kenhub.com/fr/library/anatomie/systeme-cardiovasculaire-circulatoire

[2] “Système de conduction cardiaque — Wikipédia.”. Available: https://fr.wikipedia.org/wiki/Syst%C3%A8me_de_conduction_cardiaque

[3] “Fonctionnement du cœur,” CHUV. Available: https://www.chuv.ch/fr/cardiologie/car-home/patients-et-famille/fonctionnement-du-coeur

[4] “Systole,” *Wikipédia*. Mar. 04, 2025. Available: https://fr.wikipedia.org/w/index.php?title=Systole&oldid=223554701

[5] “Diastole,” *Wikipédia*. Mar. 16, 2025.. Available: https://fr.wikipedia.org/w/index.php?title=Diastole&oldid=223952526

[6] “Cardiac cycle,” *Wikipedia*. Jun. 02, 2025. Available: https://en.wikipedia.org/w/index.php?title=Cardiac_cycle&oldid=1293580529

[7] “What Is the Cardiac Conduction System?,” Cleveland Clinic. Available: https://my.clevelandclinic.org/health/body/21648-heart-conduction-system

[8] “Électrocardiogramme (ECG),” VIDAL. Available: https://www.vidal.fr/sante/examens-tests-analyses-medicales/electrocardiogramme-ecg-holter.html

[9] “Électrocardiographie - Troubles cardiovasculaires,” Édition professionnelle du Manuel MSD. Available: https://www.msdmanuals.com/fr/professional/troubles-cardiovasculaires/tests-et-procédures-cardiovasculaires/électrocardiographie

[10] “Les arythmies (anomalies du rythme cardiaque) | Institut de cardiologie de l’Université d’Ottawa.”. Available: https://www.ottawaheart.ca/fr/maladie-du-c%C5%93ur/les-arythmies-anomalies-du-rythme-cardiaque

[11] “Électrocardiogramme (ECG),” VIDAL Available: https://www.vidal.fr/sante/examens-tests-analyses-medicales/electrocardiogramme-ecg-holter.html

[12] N. Sakli, C. Baccouch, A. Zouinkhi, H. Sakli, and M. Najjari, “Internet des objets (IoT) pour la surveillance d’ECG multi-patients,” in *Colloque sur les Objets et systèmes Connectés - COC’2021*, MARSEILLE, France: IUT d’Aix-Marseille, Mar. 2021. Available: https://hal.science/hal-03593717

[13] “Qu’est-ce que le l’Internet of Things ? | Nexus Group.”. Available: https://www.nexusgroup.com/fr/internet-des-objets-iot-definition/

[14] “After the IoT, the Internet of Medical Things (IoMT).”. Available: https://en.allisone.ai/blog/iomt-internet-of-medical-things

[15] Y. Abbassi and H. Benlahmer, “Un aperçu sur la sécurité de l’internet des objets (IOT),” in *Colloque sur les Objets et systèmes Connectés - COC’2021*, MARSEILLE, France: IUT d’Aix-Marseille, Mar. 2021. Available: https://hal.science/hal-03593723

[16] “Flask – Un des frameworks les plus populaires de Python.” Available: https://datascientest.com/avantages-et-fonctionnement-de-flask

[17] “Qu’est-ce que l’Internet des objets | L’IoT expliqué | SAP.” Available: https://www.sap.com/suisse/products/technology-platform/what-is-iot.html

[18] “SQLite : Tout savoir sur cette base de données embarquée.” Available: https://datascientest.com/sqlite-tout-savoir

[19] A. Lachtar, “Dans la discipline Ingénierie des Systèmes Informatiques (ISI)”.

[20] “HTML (HyperText Markup Language) | MDN.”. Available: https://developer.mozilla.org/fr/docs/Web/HTML

[21] “Transformer les soins de santé. L’impact des applications mobiles sur la santé | SEIDOR.” Available: https://www.seidor.com/fr-dz/blog/sante-transformer-soins-medicaux-impact-applications-mobiles

[22] “Wi-Fi,” *Wikipédia*. . Available: https://fr.wikipedia.org/w/index.php?title=Wi-Fi&oldid=226444882

[23] “L’évolution du protocole HTTP - HTTP | MDN.” Available: https://developer.mozilla.org/fr/docs/Web/HTTP/Guides/Evolution_of_HTTP

[24] “Measure analog voltage on ESP32 with ADC,” uPesy. Available: https://www.upesy.com/blogs/tutorials/measure-voltage-on-esp32-with-adc-with-arduino-code

[25] R. Guez, “API Vulnerability : comprendre les failles, défis et solutions pour sécuriser vos interfaces,” DataScientest.. Available: https://datascientest.com/api-vulnerability-tout-savoir

Annexes:

Code de L’ESP 32:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <DHT.h>
#include <Wire.h>
#include "MAX30105.h"
#include "spo2_algorithm.h"
#include "heartRate.h"

// Wi-Fi
const char* ssid = "AT-     -       -  -"     ;
const char* password = ""          ;

// Serveur Flask
const char* ecgUrl = "http://192.168.1.4:5000/ecg";
const char* sensorUrl = "http://192.168.1.4:5000/sensor_data";

// ECG
const int ecgPin = 34;

// DHT22
#define DHTPIN 27
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// MQ135
const int mq135Pin = 35;

// MAX30102
MAX30105 particleSensor;
#define MAX_SAMPLES 100
uint32_t irBuffer[MAX_SAMPLES];
uint32_t redBuffer[MAX_SAMPLES];
int32_t bufferLength = MAX_SAMPLES;
int32_t spo2;
int8_t validSpO2;
int32_t heartRate;
int8_t validHeartRate;

// Intervalles
unsigned long lastSensorSent = 0;
```

```
const unsigned long sensorInterval = 3000;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  Serial.print("Connexion au Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi connecté !");

  dht.begin();

  if (!particleSensor.begin(Wire, I2C_SPEED_STANDARD)) {
    Serial.println("MAX30102 non détecté !");
    while (1);
  }

  particleSensor.setup();
  particleSensor.setPulseAmplitudeRed(0x0A);
  particleSensor.setPulseAmplitudeIR(0x0A);
}

void loop() {
  // ECG
  int ecgValue = analogRead(ecgPin) * (3.3 / 4095.0);
  Serial.print("ECG: "); Serial.println(ecgValue);

  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(ecgUrl);
    http.addHeader("Content-Type", "application/json");

    String ecgPayload = "{\"ecg_value\":" + String(ecgValue) + "}";
    int httpResponseCode = http.POST(ecgPayload);

    if (httpResponseCode > 0) {
      Serial.println("ECG envoyé !");
    } else {
      Serial.print("Erreur ECG : "); Serial.println(httpResponseCode);
    }
    http.end();
  }

  delay(10);

  // Données toutes les 3 secondes
  if (millis() - lastSensorSent > sensorInterval) {
```

```
    lastSensorSent = millis();

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    int airQuality = analogRead(mq135Pin);

    // Lire 100 échantillons du capteur MAX30102
    for (int i = 0; i < bufferLength; i++) {
      while (particleSensor.available() == false);
      redBuffer[i] = particleSensor.getRed();
      irBuffer[i] = particleSensor.getIR();
      particleSensor.nextSample();
    }

    // Calcul BPM + SpO2
    maxim_heart_rate_and_oxygen_saturation(irBuffer, bufferLength, redBuffer,
      &spo2, &validSpO2, &heartRate, &validHeartRate);

    if (validHeartRate && validSpO2) {
      Serial.println("Données capteurs réelles :");
      Serial.print("Température : "); Serial.println(temperature);
      Serial.print("Humidité : "); Serial.println(humidity);
      Serial.print("SpO2 : "); Serial.println(spo2);
      Serial.print("BPM : "); Serial.println(heartRate);
      Serial.print("Qualité de l'air MQ135 : "); Serial.println(airQuality);

      HTTPClient http;
      http.begin(sensorUrl);
      http.addHeader("Content-Type", "application/json");

      String sensorPayload = "{";
      sensorPayload += "\"temperature\":" + String(temperature) + ",";
      sensorPayload += "\"humidity\":" + String(humidity) + ",";
      sensorPayload += "\"spo2\":" + String(spo2) + ",";
      sensorPayload += "\"bpm\":" + String(heartRate) + ",";
      sensorPayload += "\"air_quality\":" + String(airQuality);
      sensorPayload += "}";

      int httpResponseCode = http.POST(sensorPayload);
      if (httpResponseCode > 0) {
        Serial.println("Données capteurs envoyées !");
      } else {
        Serial.print("Erreur capteurs : ");
        Serial.println(httpResponseCode);
      }
      http.end();
    } else {
      Serial.println("Lecture BPM/SpO2 invalide.");
    }
  }
```

```
}
```

Code du serveur Flask et de la base de donnees SQlite :

```
from flask import Flask, request, jsonify, send_file, render_template
import sqlite3
from datetime import datetime
import os
import csv

app = Flask(__name__)
DB_FILE = 'ecg_data.db'
CSV_ECG_FILE = 'export_ecg.csv'
CSV_SENSOR_FILE = 'export_sensor.csv'

def init_db():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()

    # Table ECG
    c.execute('''
        CREATE TABLE IF NOT EXISTS ecg (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT NOT NULL,
            ecg_value REAL NOT NULL
        )
    ''')

    # Table capteurs généraux
    c.execute('''
        CREATE TABLE IF NOT EXISTS sensor_data (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT NOT NULL,
            temperature REAL,
            humidity REAL,
            spo2 INTEGER,
            bpm INTEGER,
            air_quality INTEGER
        )
    ''')

    conn.commit()
    conn.close()
    print("Base de données initialisée.")

@app.route('/')
def index():
    return render_template('index.html')

# POST données ECG
```

```
@app.route('/ecg', methods=['POST'])
def recevoir_ecg():
    data = request.get_json()
    if data and 'ecg_value' in data:
        ecg_valeur = data['ecg_value']
        timestamp = datetime.now().isoformat()

        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute("INSERT INTO ecg (timestamp, ecg_value) VALUES (?, ?)",
(timestamp, ecg_valeur))
        conn.commit()
        conn.close()

        print(f"✅ Donnée ECG reçue : {ecg_valeur}")
        return jsonify({"status": "reçu et enregistré"}), 200
    return jsonify({"error": "donnée invalide"}), 400

# POST données capteurs généraux
@app.route('/sensor_data', methods=['POST'])
def recevoir_sensor_data():
    data = request.get_json()
    if data is None:
        return jsonify({"error": "pas de données reçues"}), 400

    required_keys = ['temperature', 'humidity', 'spo2', 'bpm', 'air_quality']
    if not all(k in data for k in required_keys):
        return jsonify({"error": "champs manquants dans les données"}), 400

    timestamp = datetime.now().isoformat()
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute('''
        INSERT INTO sensor_data (timestamp, temperature, humidity, spo2, bpm,
air_quality)
        VALUES (?, ?, ?, ?, ?, ?)
    ''', (timestamp, data['temperature'], data['humidity'], data['spo2'],
data['bpm'], data['air_quality']))
    conn.commit()
    conn.close()

    print(f"Données capteurs reçues : {data}")
    return jsonify({"status": "données capteurs enregistrées"}), 200

# GET dernières 100 données ECG
@app.route('/data', methods=['GET'])
def get_ecg_data():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("SELECT ecg_value FROM ecg ORDER BY id DESC LIMIT 170")
```

```
    rows = c.fetchall()
    conn.close()
    data = [{"ecg_value": row[0]} for row in reversed(rows)]
    return jsonify(data)

# GET dernières 100 données capteurs
@app.route('/sensor_data', methods=['GET'])
def get_sensor_data():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("SELECT temperature, humidity, spo2, bpm, air_quality FROM
sensor_data ORDER BY id DESC LIMIT 100")
    rows = c.fetchall()
    conn.close()

    data = []
    for row in reversed(rows):
        data.append({
            "temperature": row[0],
            "humidity": row[1],
            "spo2": row[2],
            "bpm": row[3],
            "air_quality": row[4]
        })
    return jsonify(data)

# Export CSV ECG
@app.route('/export_ecg', methods=['GET'])
def export_csv_ecg():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("SELECT id, timestamp, ecg_value FROM ecg ORDER BY id")
    rows = c.fetchall()
    conn.close()

    with open(CSV_ECG_FILE, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['id', 'timestamp', 'ecg_value'])
        writer.writerows(rows)

    return send_file(CSV_ECG_FILE, as_attachment=True)

# Export CSV capteurs
@app.route('/export_sensor', methods=['GET'])
def export_csv_sensor():
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("SELECT id, timestamp, temperature, humidity, spo2, bpm, air_quality
FROM sensor_data ORDER BY id")
    rows = c.fetchall()
```

```
    conn.close()

    with open(CSV_SENSOR_FILE, 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['id', 'timestamp', 'temperature', 'humidity', 'spo2',
'bpm', 'air_quality'])
        writer.writerows(rows)

    return send_file(CSV_SENSOR_FILE, as_attachment=True)


if __name__ == '__main__':
    init_db()
    app.run(host='0.0.0.0', port=5000)
```

3 Code d'une partie de la page WEB

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8" />
  <title>Visualisation des Paramètres Vitaux</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      background-color: #f0f4f8;
      color: #333;
      padding: 20px;
      margin: 0;
    }
    h1, h2, h3 {
      color: #005b96;
    }
    canvas {
      background-color: #fff;
      border: 1px solid #005b96;
      max-width: 100%;
      margin-bottom: 20px;
      box-shadow: 0 0 10px rgba(0, 91, 150, 0.3);
    }
    button {
      margin: 10px 15px;
      padding: 10px 20px;
      background-color: #005b96;
      color: #fff;
      border: none;
      border-radius: 5px;
      cursor: pointer;
```

```
    font-weight: bold;
  }
  button:hover {
    background-color: #003f6b;
  }
  .sensor-values {
    display: flex;
    justify-content: center;
    gap: 25px;
    font-size: 1.1em;
    margin-bottom: 30px;
    flex-wrap: wrap;
  }
  .sensor-values div {
    background: #fff;
    padding: 10px 15px;
    border-radius: 8px;
    box-shadow: 0 0 5px rgba(0, 91, 150, 0.3);
    min-width: 120px;
    color: #005b96;
  }
  .tab {
    display: flex;
    justify-content: center;
    border-bottom: 1px solid #005b96;
    background-color: #e6ecf0;
    margin-bottom: 20px;
  }
  .tab button {
    background-color: inherit;
    border: none;
    outline: none;
    cursor: pointer;
    padding: 14px 20px;
    transition: 0.3s;
    font-size: 17px;
    color: #005b96;
  }
  .tab button:hover {
    background-color: #ccd6dd;
  }
  .tab button.active {
    background-color: #005b96;
    color: #fff;
  }
  .tabcontent {
    display: none;
    padding: 6px 12px;
    border: 1px solid #005b96;
    border-top: none;
```

```
      background-color: #fff;
    }
    .tabcontent.active {
      display: block;
    }
    #APropos {
      text-align: center;
      padding: 20px;
    }
    #APropos p {
      font-weight: bold;
      font-size: 1.2em;
      color: #005b96;
    }
    #APropos a {
      display: inline-block;
      margin-bottom: 20px;
    }
  </style>
</head>
<body>
```

Code d'une partie de l'application Flutter :

```
import 'package:flutter/material.dart';
import 'screens/dashboard_screen.dart';
import 'screens/ecg_screen.dart';
import 'screens/sensor_screen.dart';
import 'screens/about_screen.dart';
import 'screens/alert_screen.dart';

void main() {
  runApp(HealthMonitorApp());
}

class HealthMonitorApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Système de Santé Connecté',
      theme: ThemeData(primarySwatch: Colors.teal),
      home: MainNavigation(),
    );
  }
}

class MainNavigation extends StatefulWidget {
  @override
  _MainNavigationState createState() => _MainNavigationState();
```

```dart
}

class _MainNavigationState extends State<MainNavigation> {
  int _currentIndex = 0;

  final List<Widget> _screens = [
    DashboardScreen(),
    ECGScreen(),
    SensorScreen(),
    AlertScreen(),
    AboutScreen(),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: _screens[_currentIndex],
      bottomNavigationBar: BottomNavigationBar(
        currentIndex: _currentIndex,
        selectedItemColor: Colors.teal,
        unselectedItemColor: Colors.grey,
        items: const [
          BottomNavigationBarItem(icon: Icon(Icons.dashboard), label: 'Dashboard'),
          BottomNavigationBarItem(icon: Icon(Icons.favorite), label: 'ECG'),
          BottomNavigationBarItem(icon: Icon(Icons.sensors), label: 'Capteurs'),
          BottomNavigationBarItem(icon: Icon(Icons.warning), label: 'Alertes'),
          BottomNavigationBarItem(icon: Icon(Icons.info), label: 'À propos'),
        ],
        onTap: (index) => setState(() => _currentIndex = index),
      ),
    );
  }
}
```